- Shivam Sharma
- 1/23/SET/BCS/424
- 6AIMLB

House Price Prediction (Regresssion Model)

```
# ==========================================
# 1. Import Libraries
# ==========================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# ==========================================
# 2. Load Dataset as DataFrame
# ==========================================
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# ==========================================
# 3. Feature-Target Split
# ==========================================
X = df.drop("Price", axis=1)
y = df["Price"]

# ==========================================
# 4. Train-Test Split
# ==========================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ==========================================
# 5. Feature Scaling (VERY IMPORTANT)
# ------------------------------------------
```

```
# ------------------------------------------
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ==========================================
# 6. Build Multilayer Model
# ==========================================
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_tra
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)  # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

model.summary()

# ==========================================
# 7. Train Model
# ==========================================
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32,
    verbose=1
)

# ==========================================
# 8. Evaluate Model
# ==========================================
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# ==========================================
```

```
# 9. Plot Training Curve
# ==========================================
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# ==========================================
# 10. Actual vs Predicted Plot
# ==========================================
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()
```

```
Dataset Shape: (20640, 9)
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Lati
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     3
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     3
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     3
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     3
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     3

   Longitude  Price
0    -122.23  4.526
1    -122.22  3.585
2    -122.24  3.521
3    -122.25  3.413
4    -122.25  3.422
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.
  super().__init__(activity_regularizer=activity_regularizer, **kwar
```

Model: "sequential"

Layer (type)	Output Shape	P
dense (Dense)	(None, 128)	
dense_1 (Dense)	(None, 64)	
dense_2 (Dense)	(None, 32)	
dense_3 (Dense)	(None, 1)	

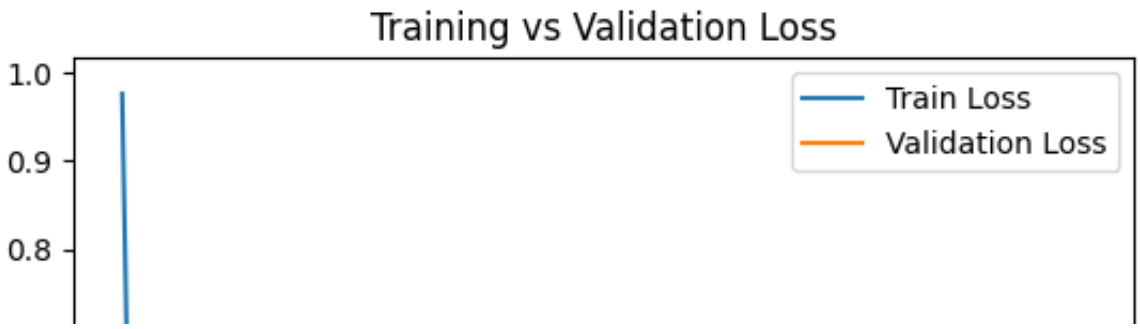
Total params: 11,521 (45.00 KB)
Trainable params: 11,521 (45.00 KB)
Non-trainable params: 0 (0.00 B)

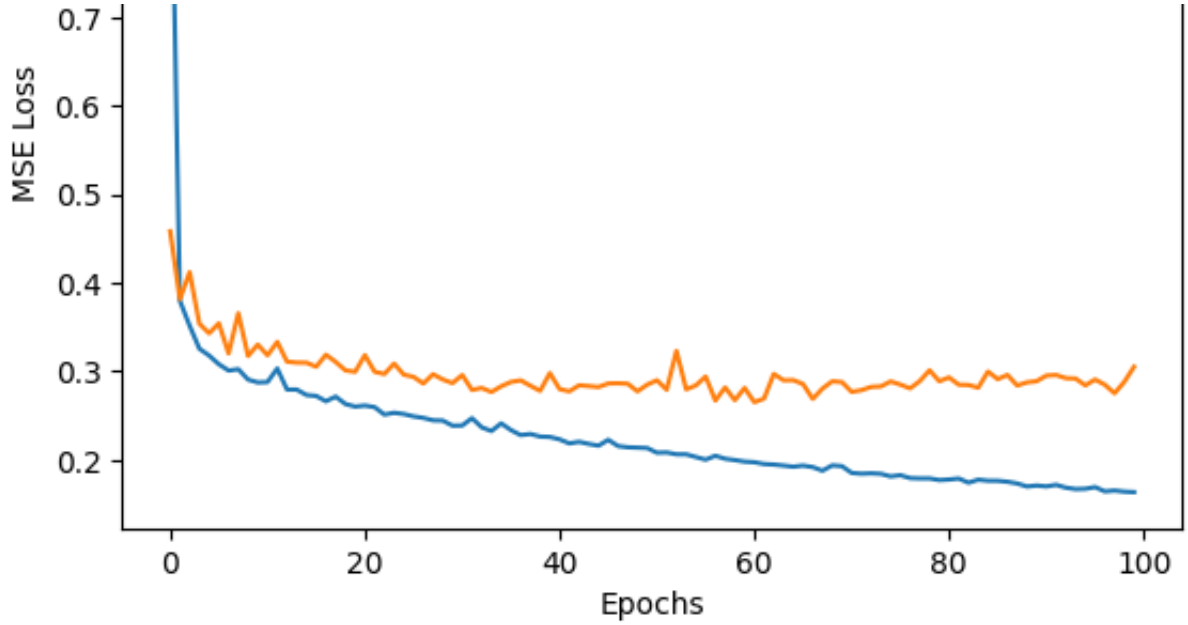
```
 Non-trainable params: 0 (0.00 B)
Epoch 1/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - loss: 1.9615 - mae: 0.765
Epoch 2/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.3759 - mae: 0.429
Epoch 3/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.3416 - mae: 0.406
Epoch 4/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.3284 - mae: 0.398
Epoch 5/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.3043 - mae: 0.385
Epoch 6/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2990 - mae: 0.380
Epoch 7/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2966 - mae: 0.371
Epoch 8/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2946 - mae: 0.378
Epoch 9/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2915 - mae: 0.372
Epoch 10/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2862 - mae: 0.367
Epoch 11/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2885 - mae: 0.369
Epoch 12/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2972 - mae: 0.367
Epoch 13/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2804 - mae: 0.368
Epoch 14/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2823 - mae: 0.361
Epoch 15/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2702 - mae: 0.356
Epoch 16/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2718 - mae: 0.358
Epoch 17/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2677 - mae: 0.354
Epoch 18/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2622 - mae: 0.349
Epoch 19/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2531 - mae: 0.347
Epoch 20/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 0.2622 - mae: 0.351
Epoch 21/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - loss: 0.2523 - mae: 0.344
Epoch 22/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2586 - mae: 0.344
Epoch 23/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2515 - mae: 0.343
Epoch 24/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2411 - mae: 0.335
Epoch 25/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2456 - mae: 0.339
Epoch 26/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2517 - mae: 0.343
Epoch 27/100
```

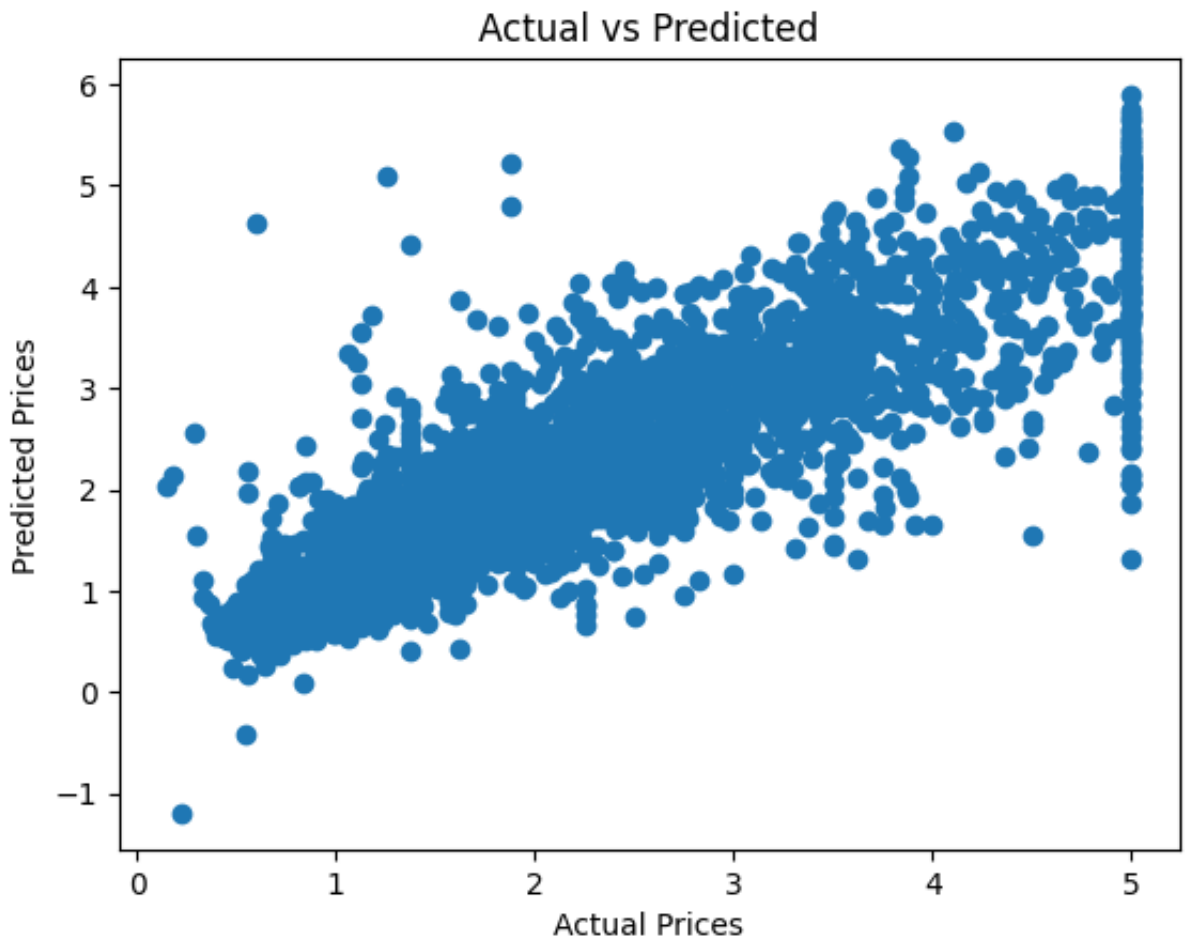
```
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2513 - mae: 0.339
Epoch 28/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2421 - mae: 0.336
Epoch 29/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2428 - mae: 0.340
Epoch 30/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2372 - mae: 0.334
Epoch 31/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2293 - mae: 0.328
Epoch 32/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2398 - mae: 0.334
Epoch 33/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2338 - mae: 0.329
Epoch 34/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2319 - mae: 0.327
Epoch 35/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2298 - mae: 0.324
Epoch 36/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2382 - mae: 0.331
Epoch 37/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2253 - mae: 0.326
Epoch 38/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2275 - mae: 0.326
Epoch 39/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2293 - mae: 0.323
Epoch 40/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2216 - mae: 0.323
Epoch 41/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2149 - mae: 0.317
Epoch 42/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2143 - mae: 0.313
Epoch 43/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2152 - mae: 0.320
Epoch 44/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2213 - mae: 0.324
Epoch 45/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2145 - mae: 0.319
Epoch 46/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2211 - mae: 0.320
Epoch 47/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2070 - mae: 0.309
Epoch 48/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2085 - mae: 0.315
Epoch 49/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2167 - mae: 0.316
Epoch 50/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2167 - mae: 0.319
Epoch 51/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2061 - mae: 0.311
Epoch 52/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2011 - mae: 0.310
Epoch 53/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2037 - mae: 0.309
Epoch 54/100
```

```
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2085 - mae: 0.313
Epoch 55/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2062 - mae: 0.309
Epoch 56/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1934 - mae: 0.301
Epoch 57/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2027 - mae: 0.306
Epoch 58/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1945 - mae: 0.303
Epoch 59/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2010 - mae: 0.307
Epoch 60/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2021 - mae: 0.308
Epoch 61/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.1975 - mae: 0.303
Epoch 62/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1926 - mae: 0.301
Epoch 63/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1928 - mae: 0.303
Epoch 64/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1888 - mae: 0.299
Epoch 65/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2016 - mae: 0.306
Epoch 66/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1935 - mae: 0.302
Epoch 67/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1889 - mae: 0.296
Epoch 68/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.1799 - mae: 0.292
Epoch 69/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1870 - mae: 0.296
Epoch 70/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1922 - mae: 0.300
Epoch 71/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1856 - mae: 0.296
Epoch 72/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1838 - mae: 0.295
Epoch 73/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1849 - mae: 0.296
Epoch 74/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1793 - mae: 0.292
Epoch 75/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.1751 - mae: 0.289
Epoch 76/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1783 - mae: 0.289
Epoch 77/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1811 - mae: 0.291
Epoch 78/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1754 - mae: 0.287
Epoch 79/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1807 - mae: 0.291
Epoch 80/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1695 - mae: 0.285
Epoch 81/100
```

```
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1846 - mae: 0.295
Epoch 82/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1738 - mae: 0.289
Epoch 83/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1761 - mae: 0.287
Epoch 84/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1707 - mae: 0.286
Epoch 85/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1756 - mae: 0.288
Epoch 86/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1765 - mae: 0.289
Epoch 87/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1831 - mae: 0.292
Epoch 88/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1762 - mae: 0.290
Epoch 89/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 0.1661 - mae: 0.282
Epoch 90/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1724 - mae: 0.287
Epoch 91/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.1647 - mae: 0.284
Epoch 92/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1673 - mae: 0.283
Epoch 93/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1635 - mae: 0.281
Epoch 94/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1663 - mae: 0.284
Epoch 95/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1720 - mae: 0.287
Epoch 96/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1737 - mae: 0.286
Epoch 97/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1521 - mae: 0.273
Epoch 98/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 4s 7ms/step - loss: 0.1589 - mae: 0.278
Epoch 99/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1600 - mae: 0.277
Epoch 100/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1657 - mae: 0.285
129/129 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2849 - mae: 0.353
129/129 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step

Test MAE: 0.3528080880641937
RMSE: 0.5334247070152569
R2 Score: 0.7828602193825904
```







Printing after Normalization 90/10 Split

```
# ==========================================
# 1. Import Libraries
# ==========================================
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# ==========================================
# 2. Load Dataset as DataFrame
# ==========================================
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# ==========================================
# 3. Feature-Target Split
# ==========================================
X = df.drop("Price", axis=1)
y = df["Price"]

# ==========================================
# 4. Train-Test Split
# ==========================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.1, random_state=42
)

# ==========================================
# 5. Feature Scaling (VERY IMPORTANT)
# ==========================================
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Convert back to DataFrame for readable printing
X_train_scaled_df = pd.DataFrame(X_train, columns=df.drop("Price",
X_test_scaled_df = pd.DataFrame(X_test, columns=df.drop("Price", ax

print("\nAfter Normalization (First 5 rows of Training Data):")
print(X_train_scaled_df.head())
```

```
print("\nAfter Normalization (First 5 rows of Test Data):")
print(X_test_scaled_df.head())

# ==========================================
# 6. Build Multilayer Model
# ==========================================
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_tr
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)  # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.005),
    loss='mse',
    metrics=['mae']
)

model.summary()

# ==========================================
# 7. Train Model
# ==========================================
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32,
    verbose=1
)

# ==========================================
# 8. Evaluate Model
# ==========================================
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# ==========================================
```

```
# 9. Plot Training Curve
# ==========================================
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# ==========================================
# 10. Actual vs Predicted Plot
# ==========================================
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()
```

```
Dataset Shape: (20640, 9)
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Lati
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     3
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     3
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     3
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     3
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     3

   Longitude  Price
0    -122.23  4.526
1    -122.22  3.585
2    -122.24  3.521
3    -122.25  3.413
4    -122.25  3.422

After Normalization (First 5 rows of Training Data):
     MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  La
0 -0.018837  0.983513 -0.066315   0.072017   -0.009699 -0.006443 -0.
1  2.185237  1.856793  0.712744  -0.137047   -0.132519 -0.044325  0.
2 -0.106583  0.110233 -0.705284  -0.025634    1.097431 -0.068481  0.
3 -0.377698 -1.001215  0.075494  -0.127940   -0.557123 -0.028728  1.
4  0.558387 -1.080604  0.358362  -0.223439   -0.014086  0.010648  0.

   Longitude
0   0.641303
1  -1.439758
2  -1.285051
3  -0.815939
4  -0.087318

After Normalization (First 5 rows of Test Data):
```

```
After Normalization (First 5 rows of Test Data):
     MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  La
0 -1.158483 -0.286713 -0.493531  -0.154095   -0.030754  0.072677  0.
1 -0.709126  0.110233 -0.156462   0.199021    0.121016 -0.036911 -0.
2 -0.207597  1.856793 -0.579091   0.183314   -0.102691 -0.157646  1.
3  0.985701 -0.921826  0.290845  -0.158389    0.243835  0.033057 -0.
4 -0.078145  0.427789  0.024015  -0.142229   -0.319380 -0.054860  0.

   Longitude
0   0.281983
1   0.057408
2  -1.429777
3   0.426709
4  -1.175258
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.
  super().__init__(activity_regularizer=activity_regularizer, **kwar
```

Model: "sequential_4"

Layer (type)	Output Shape	P
dense_16 (Dense)	(None, 128)	
dense_17 (Dense)	(None, 64)	
dense_18 (Dense)	(None, 32)	
dense_19 (Dense)	(None, 1)	

Total params: 11,521 (45.00 KB)
Trainable params: 11,521 (45.00 KB)
Non-trainable params: 0 (0.00 B)

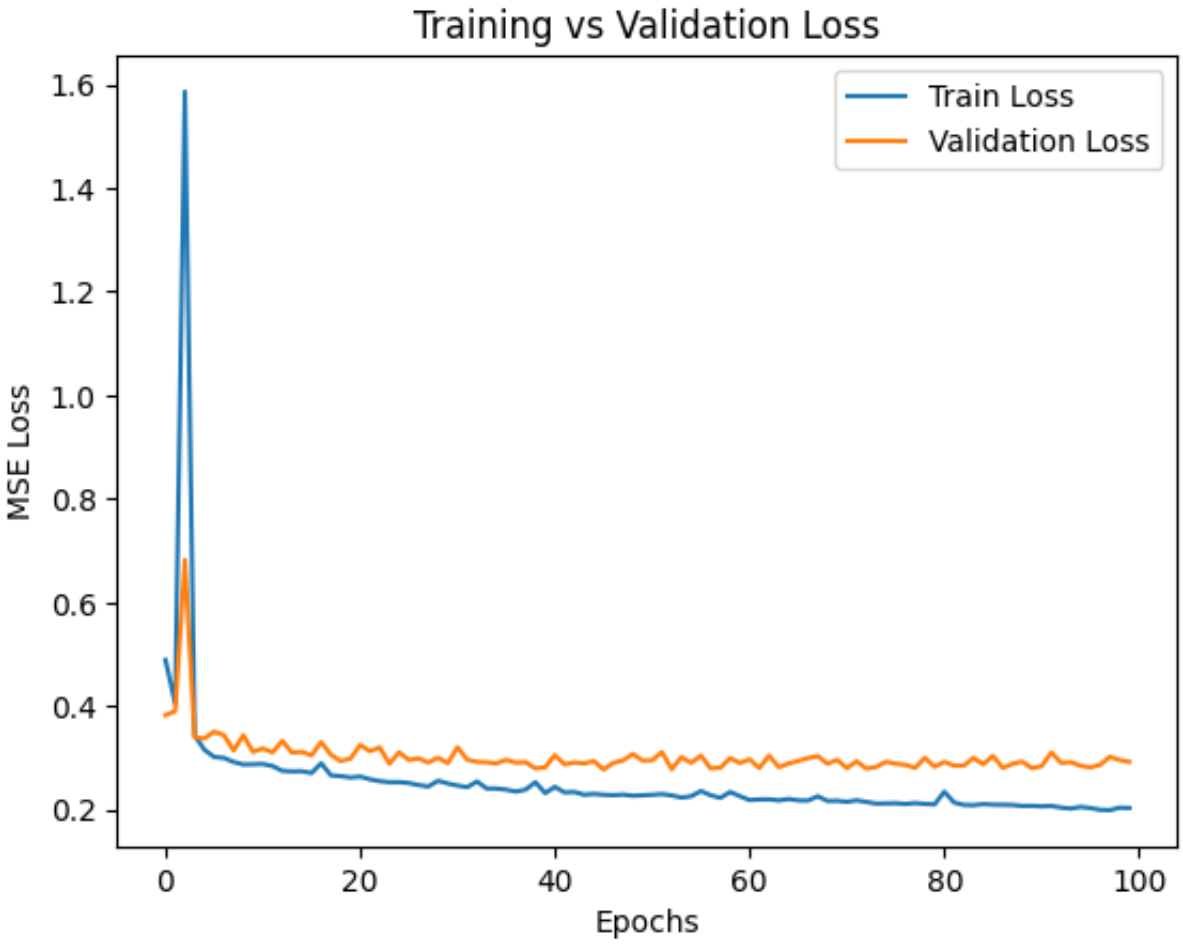
```
Epoch 1/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 0.7637 - mae: 0.591
Epoch 2/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.4206 - mae: 0.427
Epoch 3/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 2.4298 - mae: 0.424
Epoch 4/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.3691 - mae: 0.419
Epoch 5/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 0.3160 - mae: 0.385
Epoch 6/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.3007 - mae: 0.380
Epoch 7/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.3025 - mae: 0.379
Epoch 8/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2881 - mae: 0.370
Epoch 9/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2813 - mae: 0.364
Epoch 10/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2903 - mae: 0.373
Epoch 11/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2942 - mae: 0.366
Epoch 12/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2923 - mae: 0.369
```

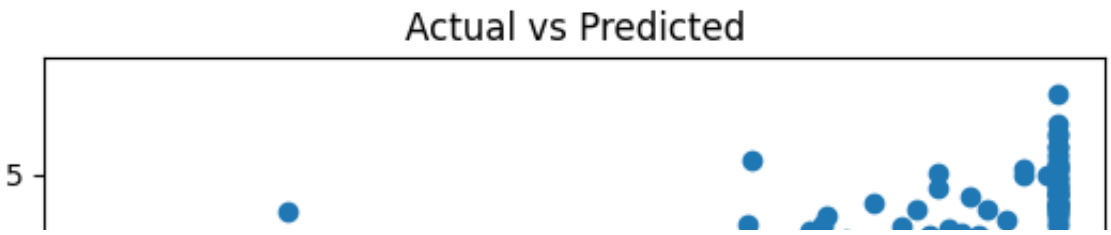
```
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2925 - mae: 0.369
Epoch 13/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2751 - mae: 0.360
Epoch 14/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2755 - mae: 0.359
Epoch 15/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2766 - mae: 0.359
Epoch 16/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2712 - mae: 0.356
Epoch 17/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.3138 - mae: 0.372
Epoch 18/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2631 - mae: 0.351
Epoch 19/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2576 - mae: 0.345
Epoch 20/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2601 - mae: 0.348
Epoch 21/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2635 - mae: 0.347
Epoch 22/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2514 - mae: 0.345
Epoch 23/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2584 - mae: 0.346
Epoch 24/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2549 - mae: 0.341
Epoch 25/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2493 - mae: 0.339
Epoch 26/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2481 - mae: 0.341
Epoch 27/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2479 - mae: 0.339
Epoch 28/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2464 - mae: 0.337
Epoch 29/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2562 - mae: 0.344
Epoch 30/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2478 - mae: 0.338
Epoch 31/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2470 - mae: 0.339
Epoch 32/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2350 - mae: 0.334
Epoch 33/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2706 - mae: 0.343
Epoch 34/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2403 - mae: 0.334
Epoch 35/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2380 - mae: 0.334
Epoch 36/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2395 - mae: 0.332
Epoch 37/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2366 - mae: 0.332
Epoch 38/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2384 - mae: 0.334
Epoch 39/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2487 - mae: 0.332
```

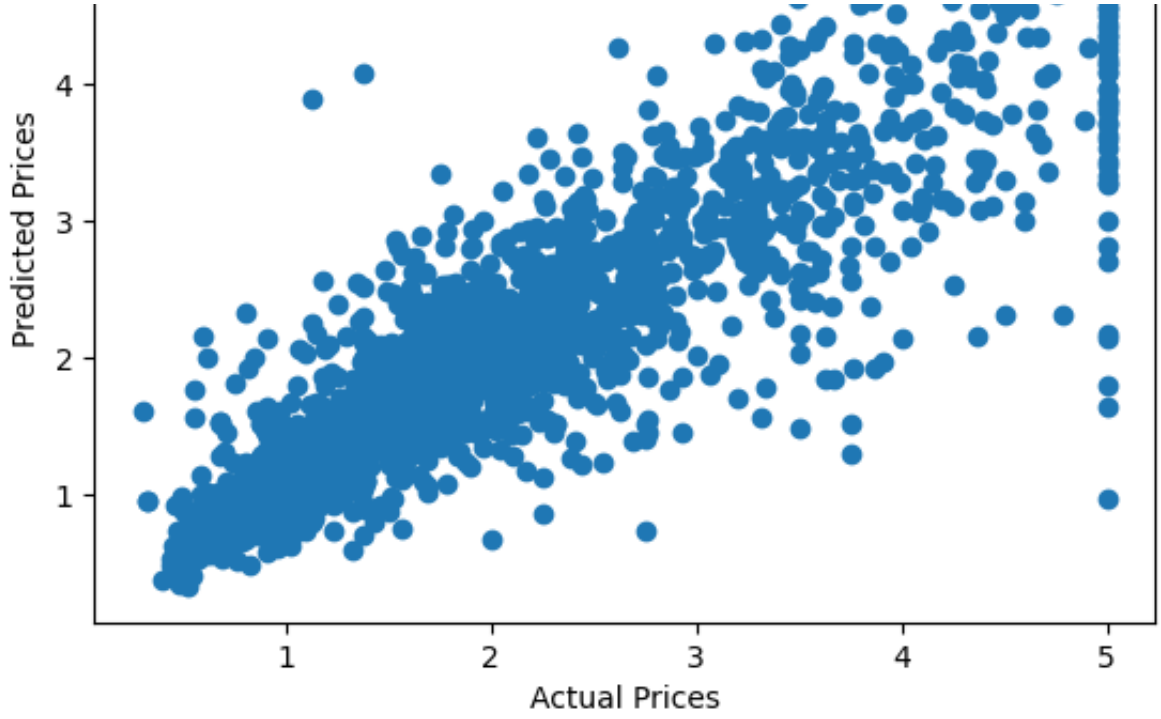
```
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2487 - mae: 0.332
Epoch 40/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2294 - mae: 0.328
Epoch 41/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2326 - mae: 0.326
Epoch 42/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2360 - mae: 0.333
Epoch 43/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2253 - mae: 0.325
Epoch 44/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2281 - mae: 0.326
Epoch 45/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2205 - mae: 0.324
Epoch 46/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2312 - mae: 0.330
Epoch 47/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 0.2342 - mae: 0.329
Epoch 48/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2239 - mae: 0.325
Epoch 49/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2225 - mae: 0.324
Epoch 50/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2392 - mae: 0.329
Epoch 51/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2346 - mae: 0.329
Epoch 52/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2316 - mae: 0.325
Epoch 53/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2302 - mae: 0.328
Epoch 54/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2229 - mae: 0.322
Epoch 55/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 0.2259 - mae: 0.326
Epoch 56/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2285 - mae: 0.326
Epoch 57/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2305 - mae: 0.329
Epoch 58/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2171 - mae: 0.320
Epoch 59/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2220 - mae: 0.321
Epoch 60/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2249 - mae: 0.323
Epoch 61/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2166 - mae: 0.318
Epoch 62/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2200 - mae: 0.320
Epoch 63/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2227 - mae: 0.321
Epoch 64/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2134 - mae: 0.318
Epoch 65/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2142 - mae: 0.319
Epoch 66/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2201 - mae: 0.321
```

```
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2201 - mae: 0.321
Epoch 67/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2216 - mae: 0.324
Epoch 68/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2136 - mae: 0.318
Epoch 69/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2228 - mae: 0.317
Epoch 70/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2133 - mae: 0.317
Epoch 71/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2211 - mae: 0.321
Epoch 72/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2120 - mae: 0.317
Epoch 73/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2082 - mae: 0.313
Epoch 74/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2059 - mae: 0.312
Epoch 75/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2076 - mae: 0.315
Epoch 76/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2064 - mae: 0.314
Epoch 77/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2084 - mae: 0.313
Epoch 78/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2027 - mae: 0.310
Epoch 79/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2076 - mae: 0.312
Epoch 80/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2171 - mae: 0.319
Epoch 81/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2077 - mae: 0.310
Epoch 82/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2174 - mae: 0.320
Epoch 83/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2061 - mae: 0.311
Epoch 84/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2056 - mae: 0.313
Epoch 85/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2142 - mae: 0.319
Epoch 86/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2101 - mae: 0.316
Epoch 87/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2036 - mae: 0.313
Epoch 88/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2005 - mae: 0.308
Epoch 89/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1987 - mae: 0.309
Epoch 90/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2044 - mae: 0.312
Epoch 91/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2075 - mae: 0.312
Epoch 92/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1978 - mae: 0.305
Epoch 93/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2047 - mae: 0.308
```

```
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2047 - mae: 0.308
Epoch 94/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2067 - mae: 0.311
Epoch 95/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1972 - mae: 0.306
Epoch 96/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2069 - mae: 0.310
Epoch 97/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2007 - mae: 0.309
Epoch 98/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2019 - mae: 0.309
Epoch 99/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1973 - mae: 0.308
Epoch 100/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2104 - mae: 0.315
65/65 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.2552 - mae: 0.3435
65/65 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step

Test MAE: 0.3500092029571533
RMSE: 0.5242571142324666
R2 Score: 0.793908523254359
```







Printing after Normalization 80/20 Split

```
# ==========================================
# 1. Import Libraries
# ==========================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# ==========================================
# 2. Load Dataset as DataFrame
# ==========================================
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# ==========================================
```

```
# 3. Feature-Target Split
# ==========================================
X = df.drop("Price", axis=1)
y = df["Price"]

# ==========================================
# 4. Train-Test Split
# ==========================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ==========================================
# 5. Feature Scaling (VERY IMPORTANT)
# ==========================================
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Convert back to DataFrame for readable printing
X_train_scaled_df = pd.DataFrame(X_train, columns=df.drop("Price",
X_test_scaled_df = pd.DataFrame(X_test, columns=df.drop("Price", ax

print("\nAfter Normalization (First 5 rows of Training Data):")
print(X_train_scaled_df.head())

print("\nAfter Normalization (First 5 rows of Test Data):")
print(X_test_scaled_df.head())

# ==========================================
# 6. Build Multilayer Model
# ==========================================
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_tr
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)  # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.010),
    loss='mse',
    metrics=['mae']
)

model.summary()
```

```
# ==========================================
# 7. Train Model
# ==========================================
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32,
    verbose=1
)

# ==========================================
# 8. Evaluate Model
# ==========================================
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# ==========================================
# 9. Plot Training Curve
# ==========================================
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# ==========================================
# 10. Actual vs Predicted Plot
# ==========================================
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()
```

```
Dataset Shape: (20640, 9)
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Lat
```

```
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Lat
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467

   Longitude  Price
0    -122.23  4.526
1    -122.22  3.585
2    -122.24  3.521
3    -122.25  3.413
4    -122.25  3.422

After Normalization (First 5 rows of Training Data):
     MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  I
0 -0.326196  0.348490 -0.174916  -0.208365    0.768276  0.051376 -1
1 -0.035843  1.618118 -0.402835  -0.128530   -0.098901 -0.117362 -(
2  0.144701 -1.952710  0.088216  -0.257538   -0.449818 -0.032280 -(
3 -1.017864  0.586545 -0.600015  -0.145156   -0.007434  0.077507 -1
4 -0.171488  1.142008  0.349007   0.086624   -0.485877 -0.068832  (

   Longitude
0   1.272587
1   0.709162
2  -0.447603
3   1.232698
4  -0.108551

After Normalization (First 5 rows of Test Data):
     MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  I
0 -1.155085 -0.286324 -0.520686  -0.171746   -0.030301  0.067408  (
1 -0.708659  0.110435 -0.165815   0.223472    0.121851 -0.036030 -(
2 -0.210402  1.856173 -0.610765   0.205892   -0.102419 -0.149989  1
3  0.975113 -0.921138  0.305115  -0.176552    0.244979  0.030011 -(
4 -0.081794  0.427842  0.024193  -0.158465   -0.319653 -0.052971  (

   Longitude
0   0.285347
1   0.060975
2  -1.424870
3   0.429943
4  -1.170581
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense
  super().__init__(activity_regularizer=activity_regularizer, **kwa
```

Model: "sequential_5"

Layer (type)	Output Shape	
dense_20 (Dense)	(None, 128)	
dense_21 (Dense)	(None, 64)	
dense_22 (Dense)	(None, 32)	

```
| dense_23 (Dense)                | (None, 1)                |
```

```
Total params: 11,521 (45.00 KB)
Trainable params: 11,521 (45.00 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 0.6666 - mae: 0.5
Epoch 2/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3951 - mae: 0.4
Epoch 3/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.4124 - mae: 0.4
Epoch 4/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.3364 - mae: 0.4
Epoch 5/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3244 - mae: 0.3
Epoch 6/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.3911 - mae: 0.4
Epoch 7/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3287 - mae: 0.3
Epoch 8/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3094 - mae: 0.3
Epoch 9/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3125 - mae: 0.3
Epoch 10/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2964 - mae: 0.3
Epoch 11/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2869 - mae: 0.3
Epoch 12/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2859 - mae: 0.3
Epoch 13/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.3104 - mae: 0.3
Epoch 14/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2886 - mae: 0.3
Epoch 15/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2966 - mae: 0.3
Epoch 16/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2833 - mae: 0.3
Epoch 17/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2805 - mae: 0.3
Epoch 18/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2720 - mae: 0.3
Epoch 19/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2653 - mae: 0.3
Epoch 20/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2702 - mae: 0.3
Epoch 21/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2667 - mae: 0.3
Epoch 22/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2711 - mae: 0.3
Epoch 23/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2564 - mae: 0.3
Epoch 24/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2587 - mae: 0.3
Epoch 25/100
```

```
Epoch 25/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2616 - mae: 0.3
Epoch 26/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2545 - mae: 0.3
Epoch 27/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2661 - mae: 0.3
Epoch 28/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2507 - mae: 0.3
Epoch 29/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2606 - mae: 0.3
Epoch 30/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2458 - mae: 0.3
Epoch 31/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2578 - mae: 0.3
Epoch 32/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2550 - mae: 0.3
Epoch 33/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2554 - mae: 0.3
Epoch 34/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2581 - mae: 0.3
Epoch 35/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2466 - mae: 0.3
Epoch 36/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2397 - mae: 0.3
Epoch 37/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2488 - mae: 0.3
Epoch 38/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2387 - mae: 0.3
Epoch 39/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2524 - mae: 0.3
Epoch 40/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2422 - mae: 0.3
Epoch 41/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2418 - mae: 0.3
Epoch 42/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2419 - mae: 0.3
Epoch 43/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2496 - mae: 0.3
Epoch 44/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2441 - mae: 0.3
Epoch 45/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2496 - mae: 0.3
Epoch 46/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2379 - mae: 0.3
Epoch 47/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2374 - mae: 0.3
Epoch 48/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2572 - mae: 0.3
Epoch 49/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2423 - mae: 0.3
Epoch 50/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2446 - mae: 0.3
Epoch 51/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2415 - mae: 0.3
Epoch 52/100
```

```
Epoch 52/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2273 - mae: 0.32
Epoch 53/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2310 - mae: 0.33
Epoch 54/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2295 - mae: 0.33
Epoch 55/100
 45/465 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.2877 - mae: 0.36
```

Printing after Normalization 70/30 Split

```
# ==========================================
# 1. Import Libraries
# ==========================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# ==========================================
# 2. Load Dataset as DataFrame
# ==========================================
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# ==========================================
# 3. Feature-Target Split
# ==========================================
X = df.drop("Price", axis=1)
y = df["Price"]

# ==========================================
# 4. Train-Test Split
# ==========================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
# ==========================================
# 5. Feature Scaling (VERY IMPORTANT)
# ==========================================
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Convert back to DataFrame for readable printing
X_train_scaled_df = pd.DataFrame(X_train, columns=df.drop("Price",
X_test_scaled_df = pd.DataFrame(X_test, columns=df.drop("Price", ax

print("\nAfter Normalization (First 5 rows of Training Data):")
print(X_train_scaled_df.head())

print("\nAfter Normalization (First 5 rows of Test Data):")
print(X_test_scaled_df.head())

# ==========================================
# 6. Build Multilayer Model
# ==========================================
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_tr
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)  # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.015),
    loss='mse',
    metrics=['mae']
)

model.summary()

# ==========================================
# 7. Train Model
# ==========================================
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32,
    verbose=1
)
```

```
# ==========================================
# 8. Evaluate Model
# ==========================================
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# ==========================================
# 9. Plot Training Curve
# ==========================================
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# ==========================================
# 10. Actual vs Predicted Plot
# ==========================================
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()
```

```
Dataset Shape: (20640, 9)
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Lat
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467

   Longitude  Price
0    -122.23  4.526
1    -122.22  3.585
2    -122.24  3.521
3    -122.25  3.413
4    -122.25  3.422
```

```
After Normalization (First 5 rows of Training Data):
     MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  I
0  0.133506  0.509357  0.181060  -0.273850   -0.184117 -0.010825 -(
1 -0.532218 -0.679873 -0.422630  -0.047868   -0.376191 -0.089316 -1
2  0.170990 -0.362745  0.073128  -0.242600   -0.611240 -0.044800 -(
3 -0.402916 -1.155565  0.175848  -0.008560   -0.987495 -0.075230  1
4 -0.299285  1.857152 -0.259598  -0.070993    0.086015 -0.066357  (

   Longitude
0   0.780934
1   1.245270
2  -0.277552
3  -0.706938
4  -1.430902

After Normalization (First 5 rows of Test Data):
     MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  I
0 -1.152689 -0.283463 -0.507818  -0.169278   -0.031510  0.061278  (
1 -0.706406  0.112947 -0.162520   0.213327    0.120220 -0.035516 -(
2 -0.208307  1.857152 -0.595467   0.196308   -0.103428 -0.142154  1
3  0.976831 -0.917719  0.295707  -0.173931    0.243007  0.026283 -(
4 -0.079740  0.430075  0.022362  -0.156421   -0.320060 -0.051368  (

   Longitude
0   0.286641
1   0.061963
2  -1.425909
3   0.431434
4  -1.171273
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense
  super().__init__(activity_regularizer=activity_regularizer, **kwa
```

Model: "sequential_2"

Layer (type)	Output Shape	
dense_8 (Dense)	(None, 128)	
dense_9 (Dense)	(None, 64)	
dense_10 (Dense)	(None, 32)	
dense_11 (Dense)	(None, 1)	

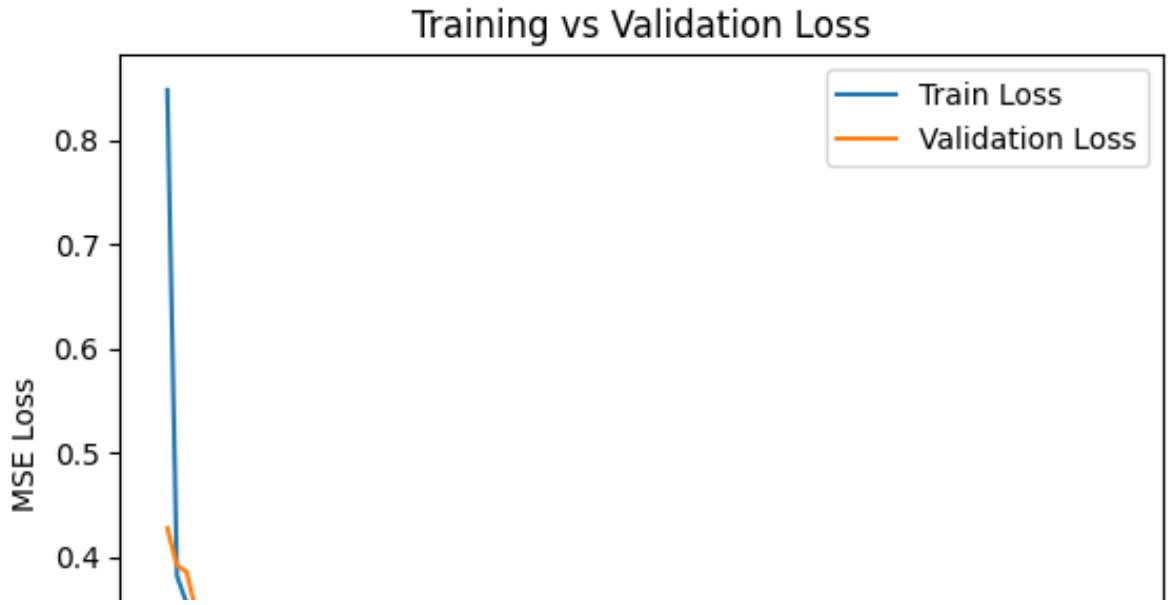
```
 Total params: 11,521 (45.00 KB)
 Trainable params: 11,521 (45.00 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 1.6104 - mae: 0.8
Epoch 2/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.4024 - mae: 0.4
Epoch 3/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3531 - mae: 0.4
Epoch 4/100
```

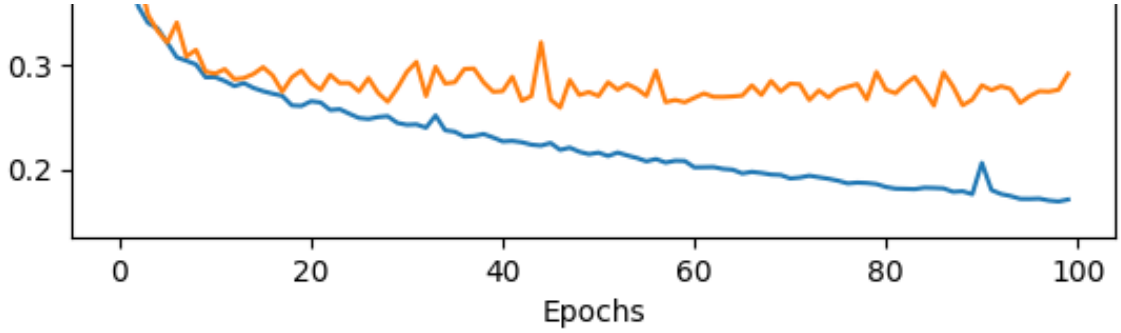
```
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3415 - mae: 0.4
Epoch 5/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3207 - mae: 0.3
Epoch 6/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3282 - mae: 0.3
Epoch 7/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3169 - mae: 0.3
Epoch 8/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3052 - mae: 0.3
Epoch 9/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2989 - mae: 0.3
Epoch 10/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2853 - mae: 0.3
Epoch 11/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2875 - mae: 0.3
Epoch 12/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2950 - mae: 0.3
Epoch 13/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2936 - mae: 0.3
Epoch 14/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2826 - mae: 0.3
Epoch 15/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2757 - mae: 0.3
Epoch 16/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2685 - mae: 0.3
Epoch 17/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2659 - mae: 0.3
Epoch 18/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2707 - mae: 0.3
Epoch 19/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2674 - mae: 0.3
Epoch 20/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2599 - mae: 0.3
Epoch 21/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2628 - mae: 0.3
Epoch 22/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2625 - mae: 0.3
Epoch 23/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2717 - mae: 0.3
Epoch 24/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2540 - mae: 0.3
Epoch 25/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2446 - mae: 0.3
Epoch 26/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2543 - mae: 0.3
Epoch 27/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2347 - mae: 0.3
Epoch 28/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2609 - mae: 0.3
Epoch 29/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 0.2399 - mae: 0.3
Epoch 30/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2265 - mae: 0.3
Epoch 31/100
```

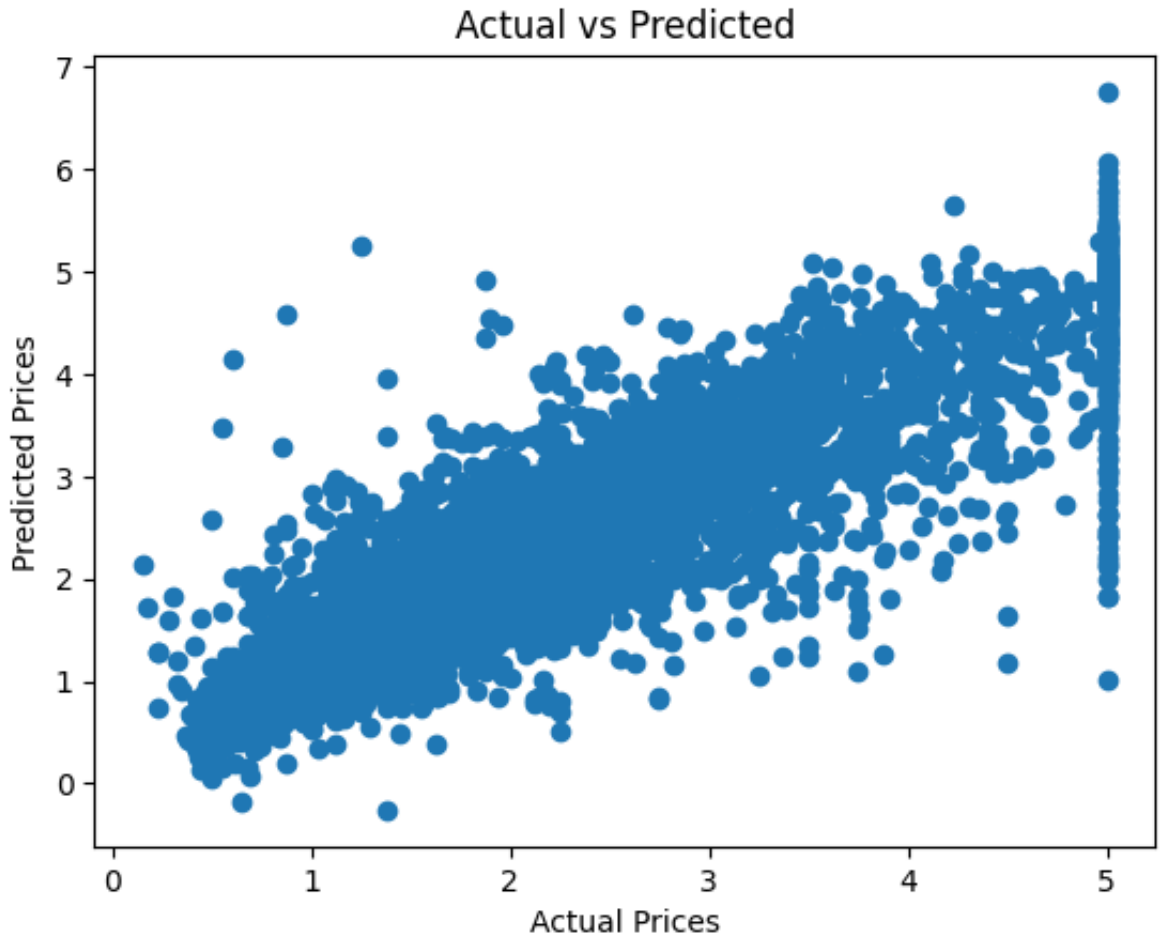
```
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2395 - mae: 0.3
Epoch 32/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2389 - mae: 0.3
Epoch 33/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2420 - mae: 0.3
Epoch 34/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2374 - mae: 0.3
Epoch 35/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2388 - mae: 0.3
Epoch 36/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2340 - mae: 0.3
Epoch 37/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2292 - mae: 0.3
Epoch 38/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2256 - mae: 0.3
Epoch 39/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2377 - mae: 0.3
Epoch 40/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2348 - mae: 0.3
Epoch 41/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2261 - mae: 0.3
Epoch 42/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2207 - mae: 0.3
Epoch 43/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2117 - mae: 0.3
Epoch 44/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2124 - mae: 0.3
Epoch 45/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2199 - mae: 0.3
Epoch 46/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2262 - mae: 0.3
Epoch 47/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2224 - mae: 0.3
Epoch 48/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2151 - mae: 0.3
Epoch 49/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2174 - mae: 0.3
Epoch 50/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2111 - mae: 0.3
Epoch 51/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2034 - mae: 0.3
Epoch 52/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2142 - mae: 0.3
Epoch 53/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2117 - mae: 0.3
Epoch 54/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2166 - mae: 0.3
Epoch 55/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2007 - mae: 0.3
Epoch 56/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.2030 - mae: 0.3
Epoch 57/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2025 - mae: 0.3
Epoch 58/100
```

```
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1999 - mae: 0.3(
Epoch 59/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2027 - mae: 0.3(
Epoch 60/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2079 - mae: 0.3:
Epoch 61/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2008 - mae: 0.3(
Epoch 62/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2008 - mae: 0.3(
Epoch 63/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2023 - mae: 0.3(
Epoch 64/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2061 - mae: 0.3(
Epoch 65/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1926 - mae: 0.3(
Epoch 66/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.1921 - mae: 0.2!
Epoch 67/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.1869 - mae: 0.2!
Epoch 68/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1983 - mae: 0.3(
Epoch 69/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1901 - mae: 0.2!
Epoch 70/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1962 - mae: 0.3(
Epoch 71/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1773 - mae: 0.2!
Epoch 72/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1826 - mae: 0.2!
Epoch 73/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1948 - mae: 0.3(
Epoch 74/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1853 - mae: 0.2!
Epoch 75/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1852 - mae: 0.2!
Epoch 76/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.1868 - mae: 0.2!
Epoch 77/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1836 - mae: 0.2!
Epoch 78/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1775 - mae: 0.2!
Epoch 79/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1900 - mae: 0.2!
Epoch 80/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1868 - mae: 0.2!
Epoch 81/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1826 - mae: 0.2!
Epoch 82/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1781 - mae: 0.2!
Epoch 83/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1746 - mae: 0.2!
Epoch 84/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1716 - mae: 0.2!
Epoch 85/100
```

```
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1849 - mae: 0.2
Epoch 86/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1691 - mae: 0.2
Epoch 87/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1783 - mae: 0.2
Epoch 88/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1741 - mae: 0.2
Epoch 89/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1765 - mae: 0.2
Epoch 90/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1744 - mae: 0.2
Epoch 91/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1710 - mae: 0.2
Epoch 92/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1956 - mae: 0.2
Epoch 93/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1876 - mae: 0.2
Epoch 94/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1737 - mae: 0.2
Epoch 95/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1702 - mae: 0.2
Epoch 96/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1719 - mae: 0.2
Epoch 97/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1699 - mae: 0.2
Epoch 98/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1621 - mae: 0.2
Epoch 99/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1711 - mae: 0.2
Epoch 100/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1685 - mae: 0.2
194/194 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.2891 - mae: 0.3
194/194 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step

Test MAE: 0.36201706528663635
RMSE: 0.5325324213337924
R2 Score: 0.7839374369717553
```







Lab 8 | 12 March 2026

- Shivam Sharma
- 1/23/SET/BCS/424
- 6AIMLB

Default Code

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model
model = Sequential([
    Dense(10, activation='softmax', input_shape=(784,))
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train, y_train, epochs=10, batch_size=64)

test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)
```

Model: "sequential_12"

Layer (type)	Output Shape	P
dense_15 (Dense)	(None, 10)	

Total params: 7,850 (30.66 KB)
Trainable params: 7,850 (30.66 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7686 - loss:
```

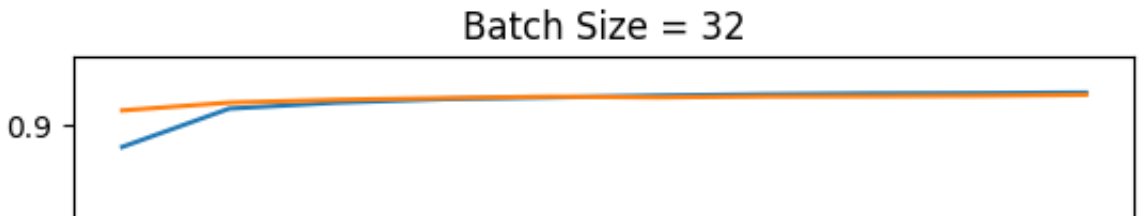
```
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9078 - loss:
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9189 - loss:
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9195 - loss:
Epoch 5/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9218 - loss:
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9235 - loss:
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9267 - loss:
Epoch 8/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9281 - loss:
Epoch 9/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9292 - loss:
Epoch 10/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9281 - loss:
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9158 - loss:
```

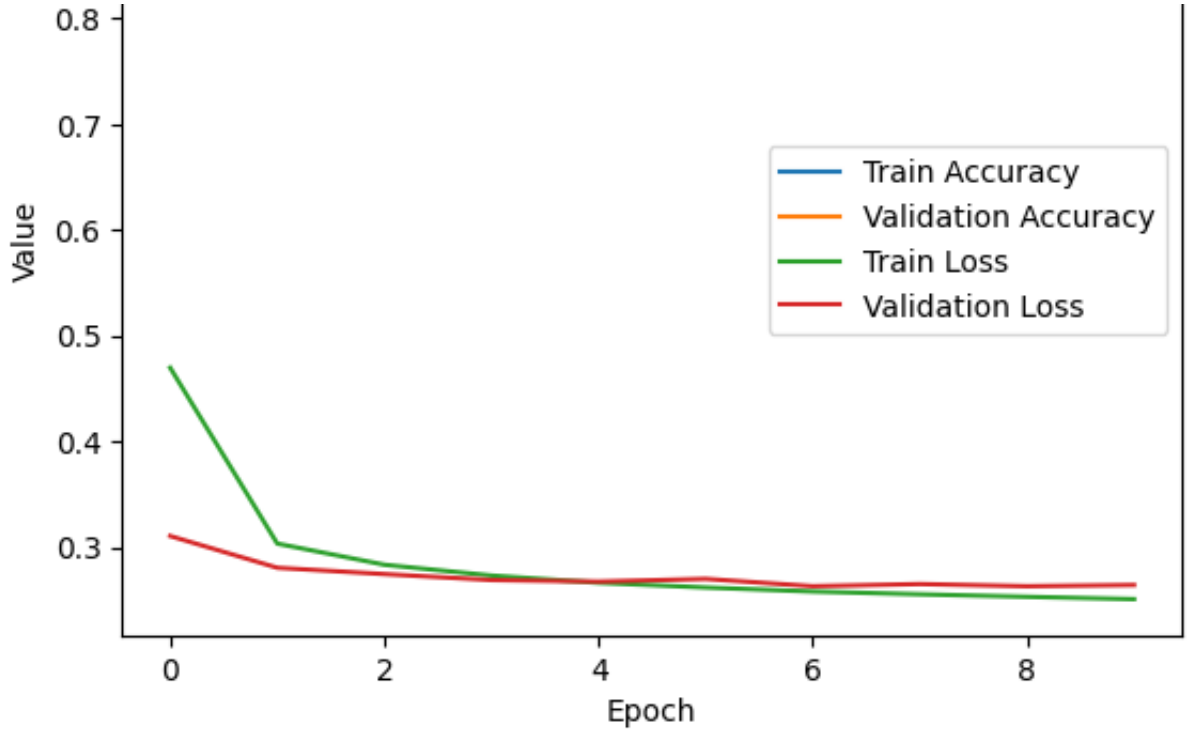
Default + Change the batch size

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Batch sizes to test
batch_sizes = [32, 64, 128, 256]
```

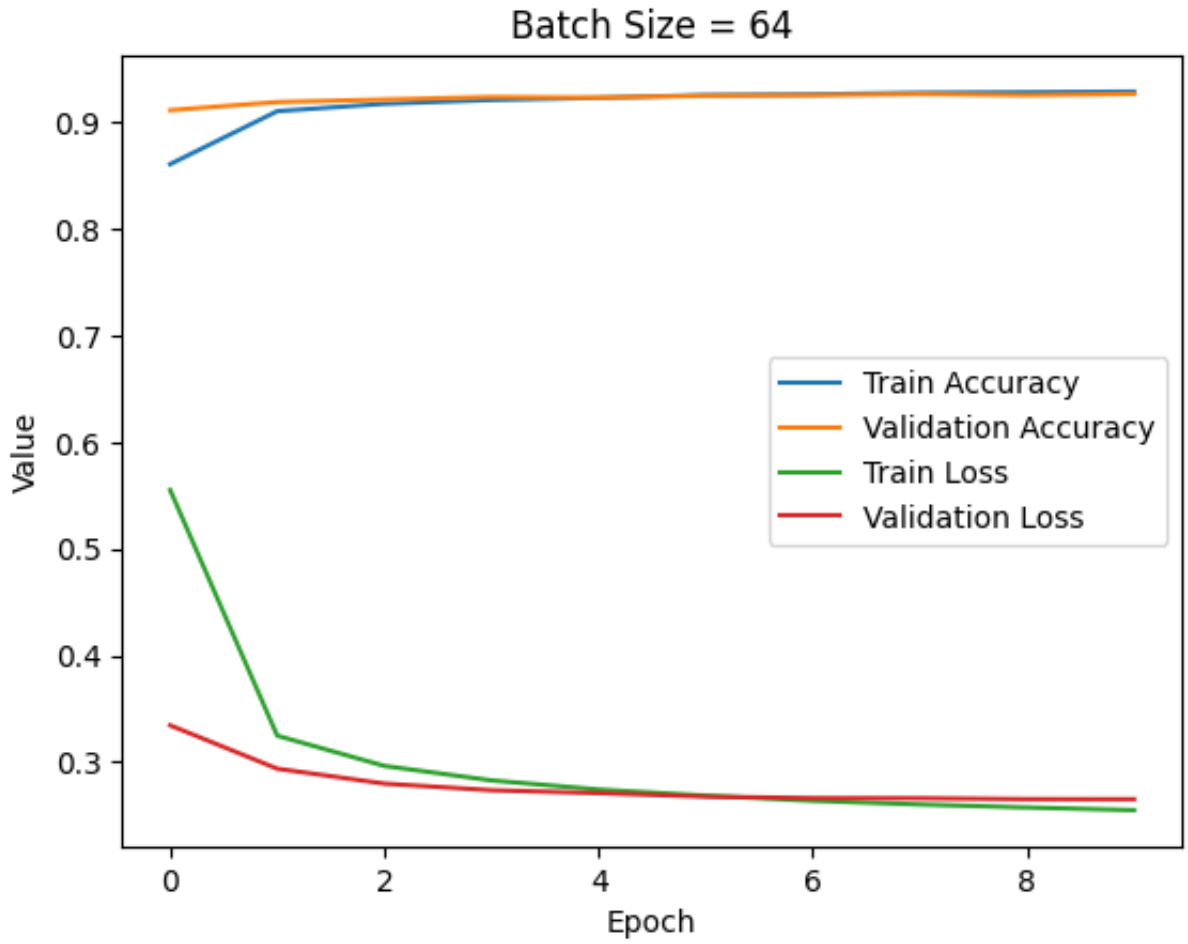
```
for batch in batch_sizes:

    print("Training with batch size:", batch)

    # Model
    model = Sequential([
        Dense(10, activation='softmax', input_shape=(784,))
    ])

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    # Train model
    history = model.fit(
        X_train,
        y_train,
        epochs=10,
        batch_size=batch,
        validation_data=(X_test, y_test),
        verbose=0
    )

    # Plot graph for this batch
    plt.figure()

    plt.plot(history.history['accuracy'], label='Train Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Acc
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')

    plt.title(f'Batch Size = {batch}')
    plt.xlabel("Epoch")
    plt.ylabel("Value")
    plt.legend()

    plt.show()
```

```
Training with batch size: 32
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.
  super().__init__(activity_regularizer=activity_regularizer, **kwar
```

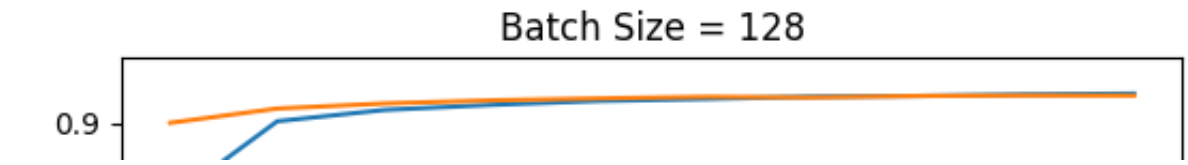


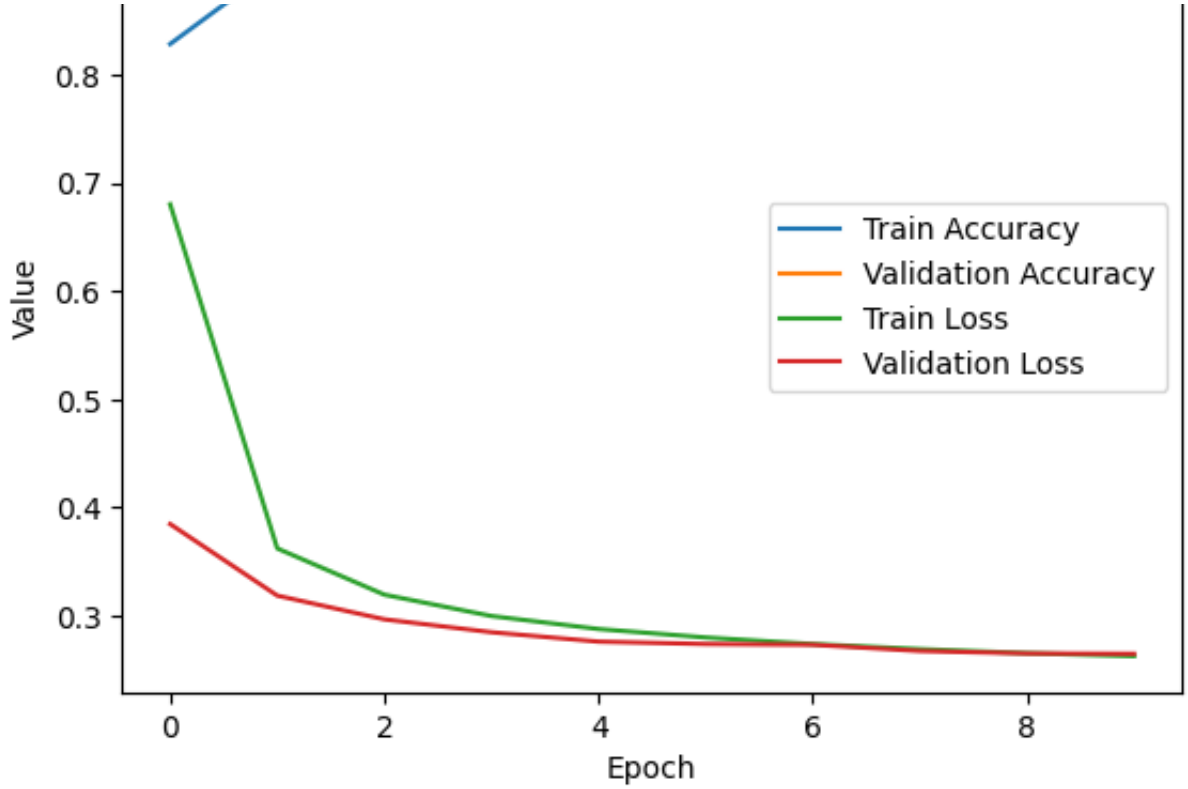



```
Training with batch size: 64
```

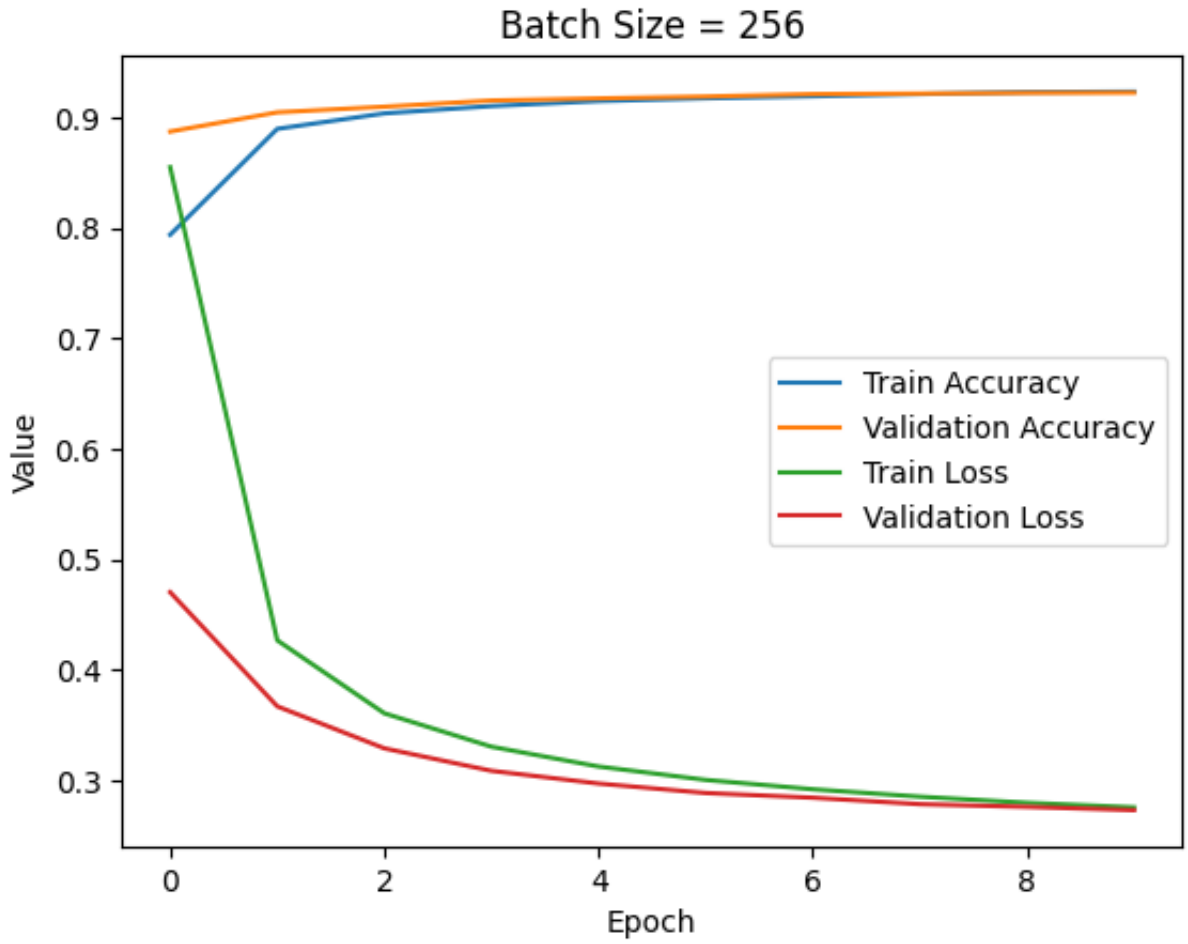



```
Training with batch size: 128
```






```
Training with batch size: 256
```

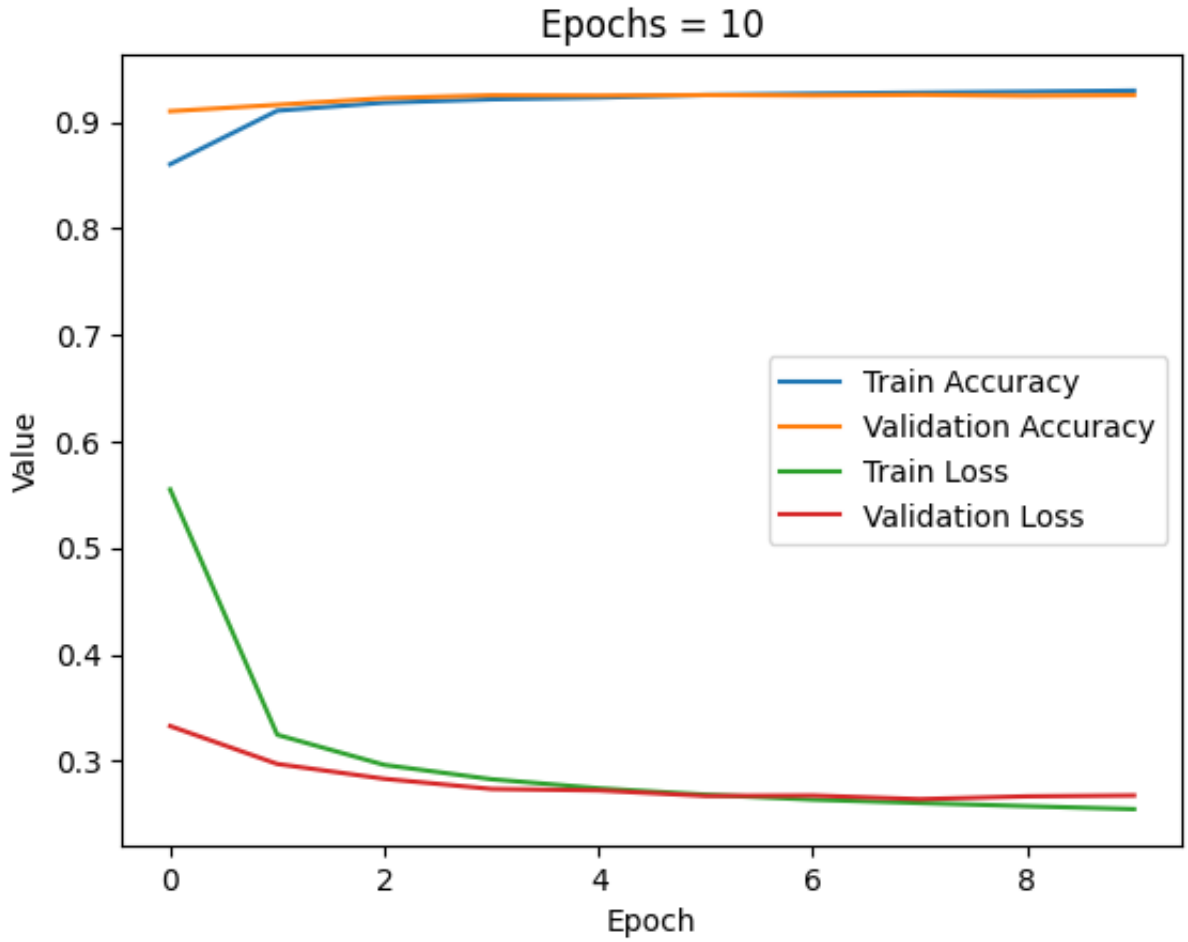


Default + Change no. of Epochs

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten images
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Epoch values to test
epoch_values = [10, 15, 20]

for ep in epoch_values:

    print("Training with epochs:", ep)

    # Model
    model = Sequential([
        Dense(10, activation='softmax', input_shape=(784,))
    ])

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    # Train model
    history = model.fit(
        X_train,
        y_train,
```

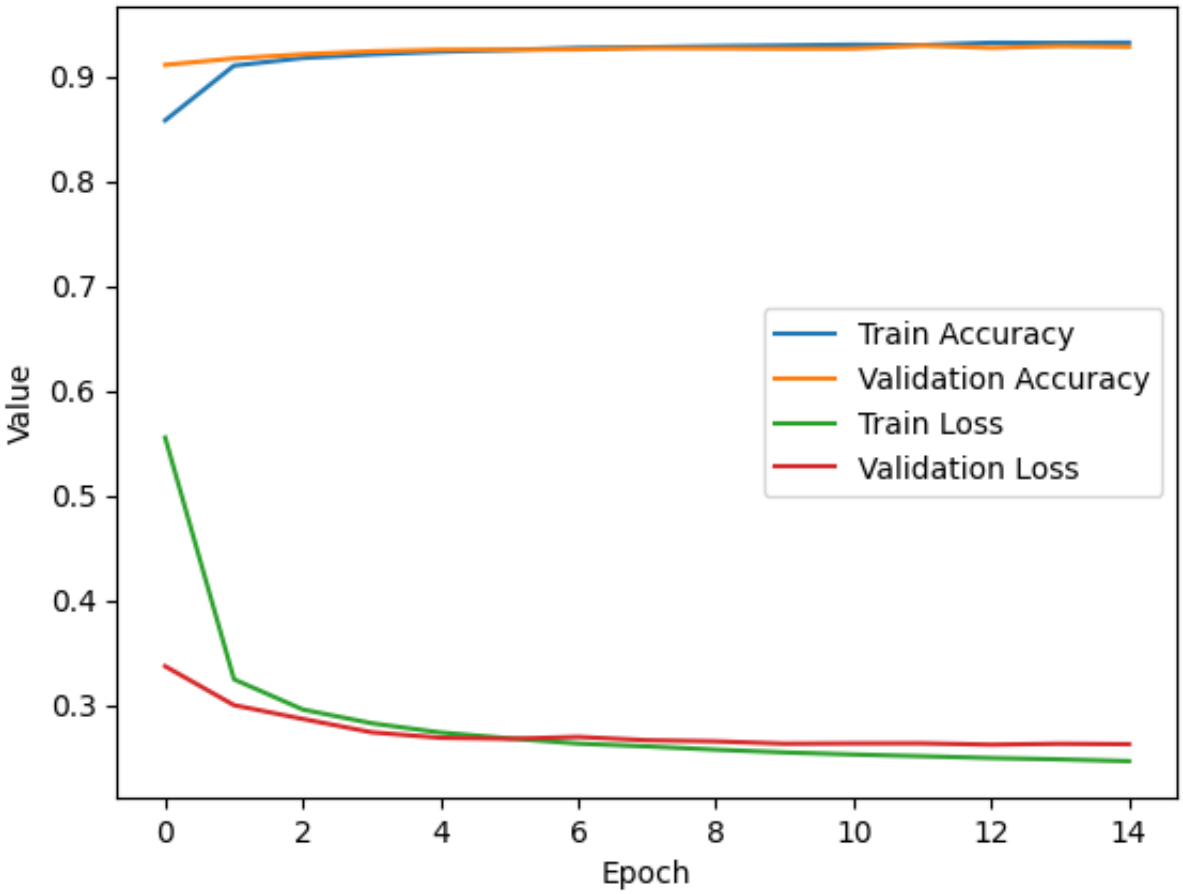
```
        epochs=ep,
        batch_size=64,
        validation_data=(X_test, y_test),
        verbose=0
    )

    # Plot graph
    plt.figure()

    plt.plot(history.history['accuracy'], label='Train Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Acc
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')

    plt.title(f'Epochs = {ep}')
    plt.xlabel("Epoch")
    plt.ylabel("Value")
    plt.legend()

    plt.show()
```

```
Training with epochs: 10
```

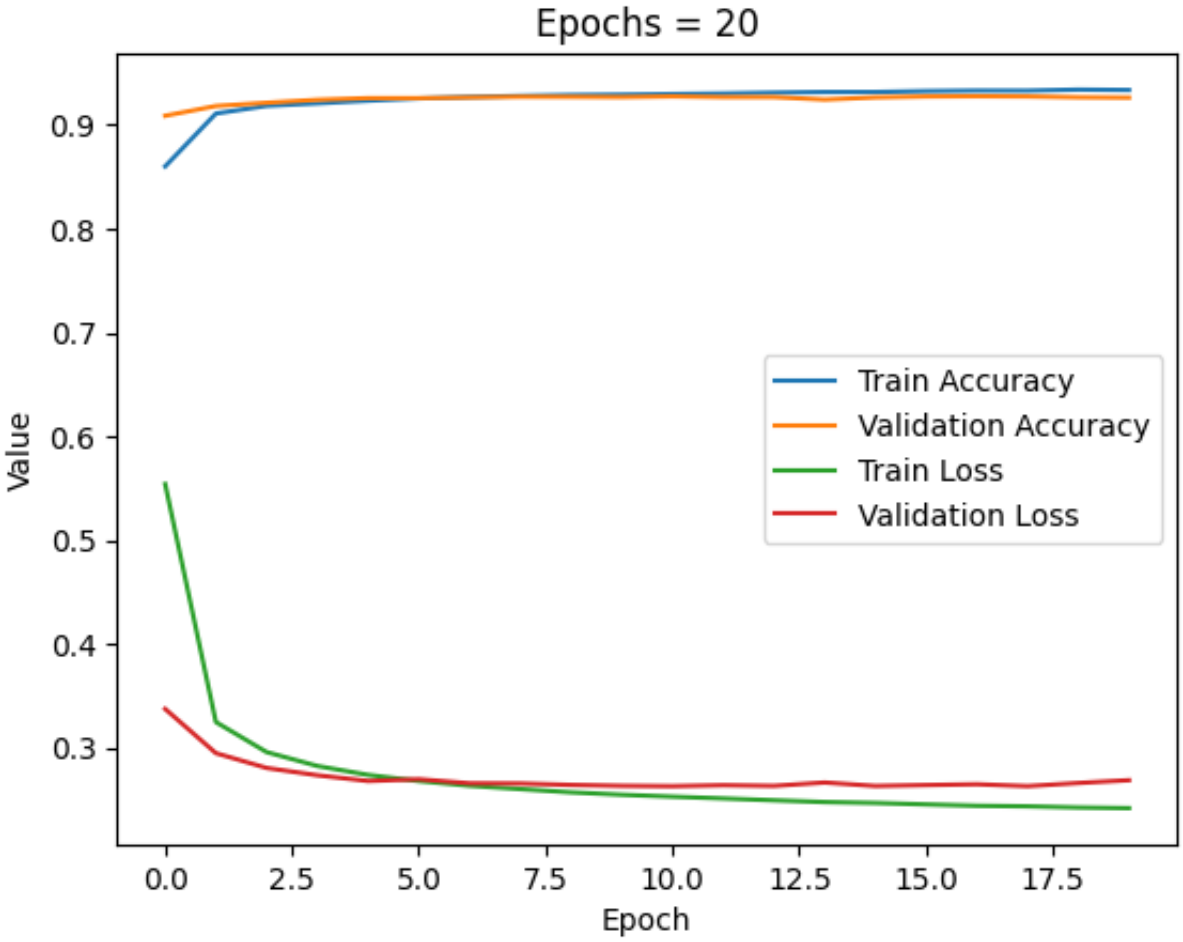



```
Training with epochs: 15
```

Epochs = 15




```
Training with epochs: 20
```



Default + Accuracy and Loss Curves

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images (28x28 → 784)
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# 5. Build model
model = Sequential([
    Dense(10, activation='softmax', input_shape=(784,))
])

# 6. Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 7. Model summary
model.summary()

# 8. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=10,
```

```
    batch_size=64,
    validation_data=(X_test, y_test)
)

# 9. Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# 10. Plot Accuracy Curve
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

# 11. Plot Loss Curve
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```

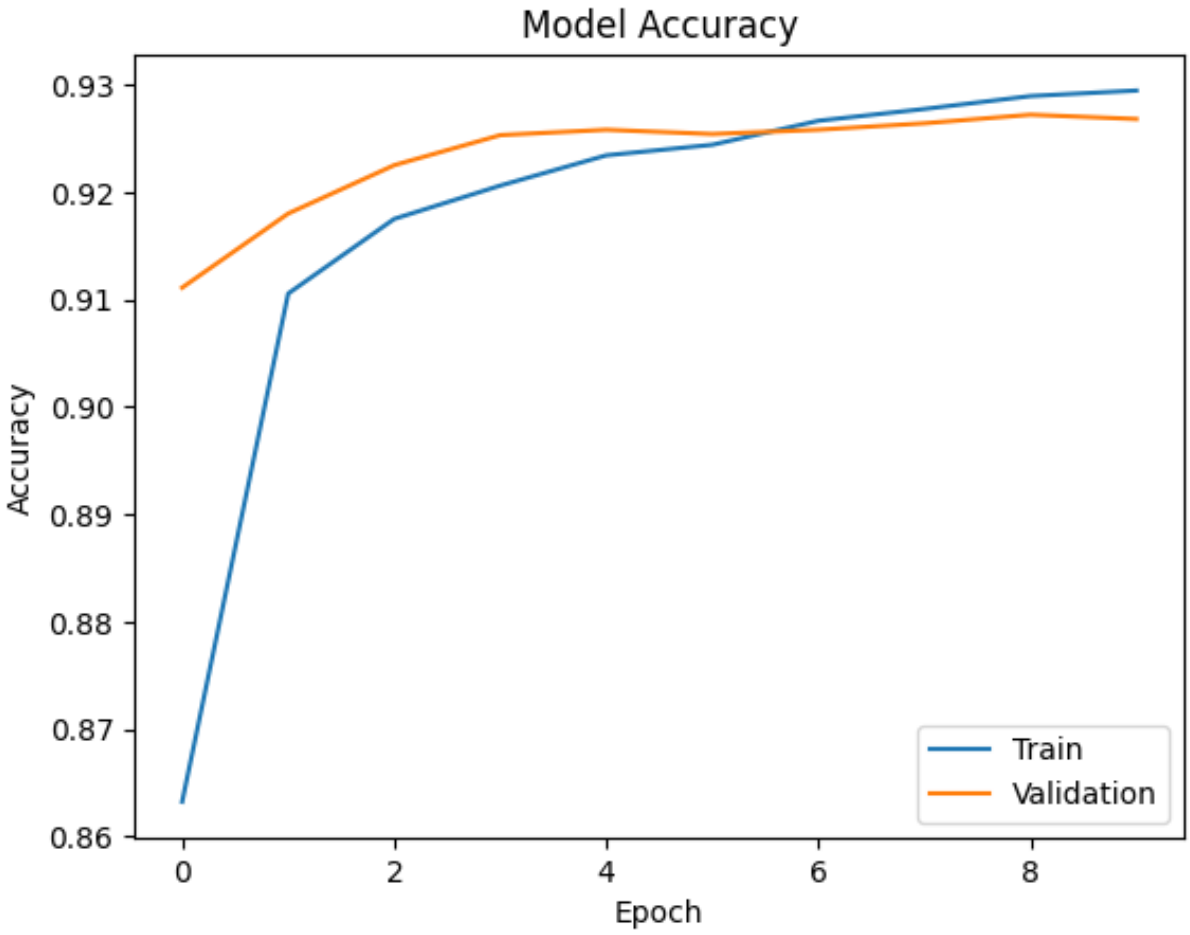
Model: "sequential_2"

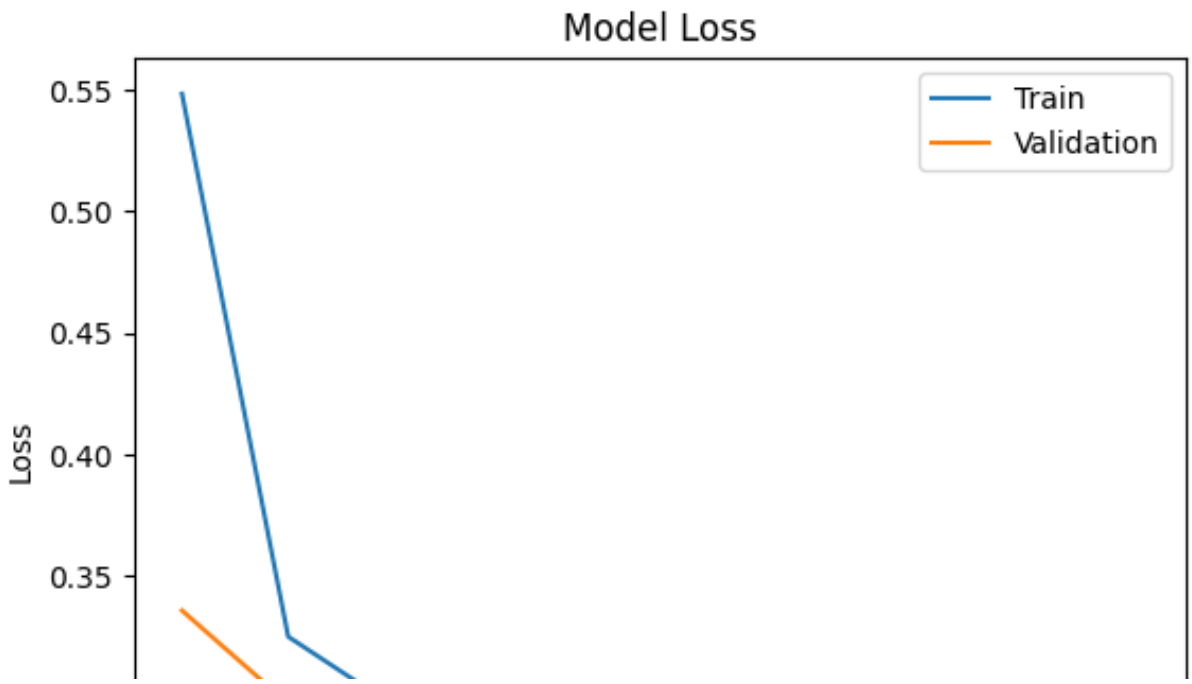
Layer (type)	Output Shape	P
dense_2 (Dense)	(None, 10)	

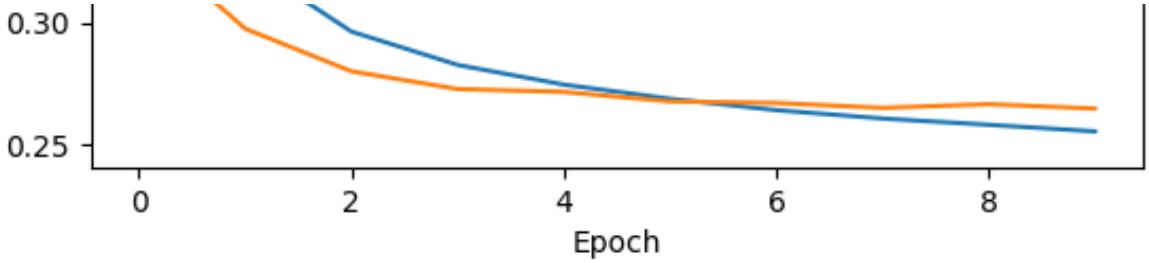
Total params: 7,850 (30.66 KB)
Trainable params: 7,850 (30.66 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.7866 - loss:
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9088 - loss:
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9173 - loss:
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9208 - loss:
Epoch 5/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9236 - loss:
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9245 - loss:
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9267 - loss:
```

```
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9267 - loss:
Epoch 8/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9274 - loss:
Epoch 9/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9271 - loss:
Epoch 10/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9292 - loss:
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9163 - loss:
Test accuracy: 0.926800012588501
```







ADD Next Layer using Reloc

32 hidden layers

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images (28x28 → 784)
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# 5. Build model (Added hidden layer with 32 neurons)
model = Sequential([
    Dense(32, activation='relu', input_shape=(784,)),   # Hidden la
    Dense(10, activation='softmax')                     # Output la
])

# 6. Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
```

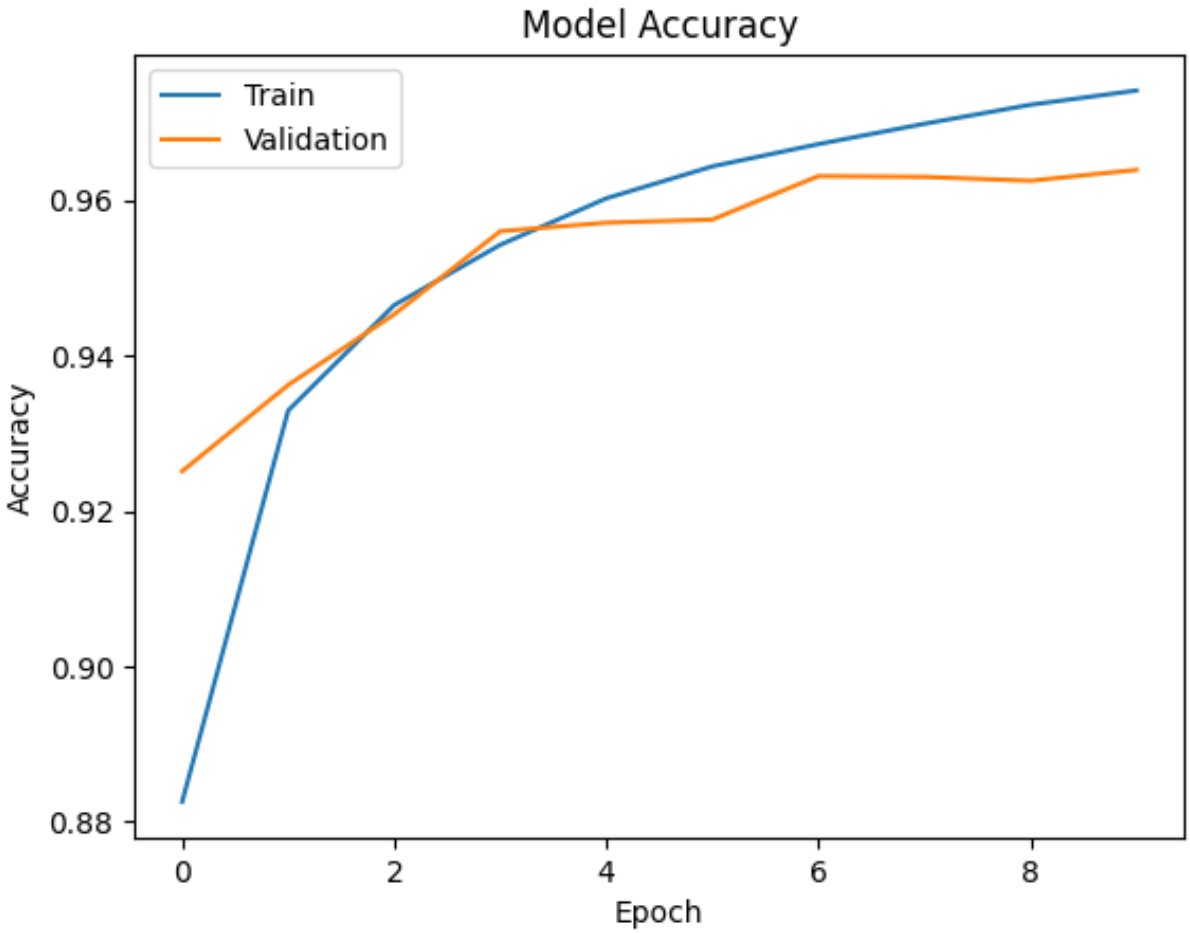
```
    metrics=['accuracy']
)

# 7. Model summary
model.summary()

# 8. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)

# 9. Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# 10. Plot Accuracy Curve
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

# 11. Plot Loss Curve
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```

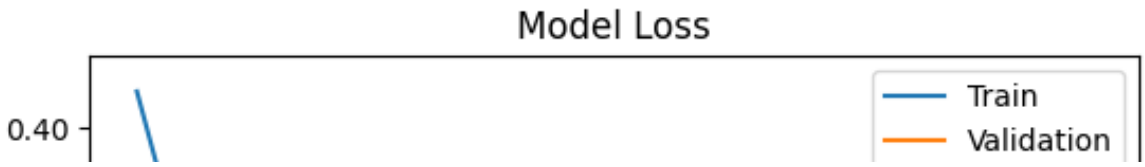
Model: "sequential_10"

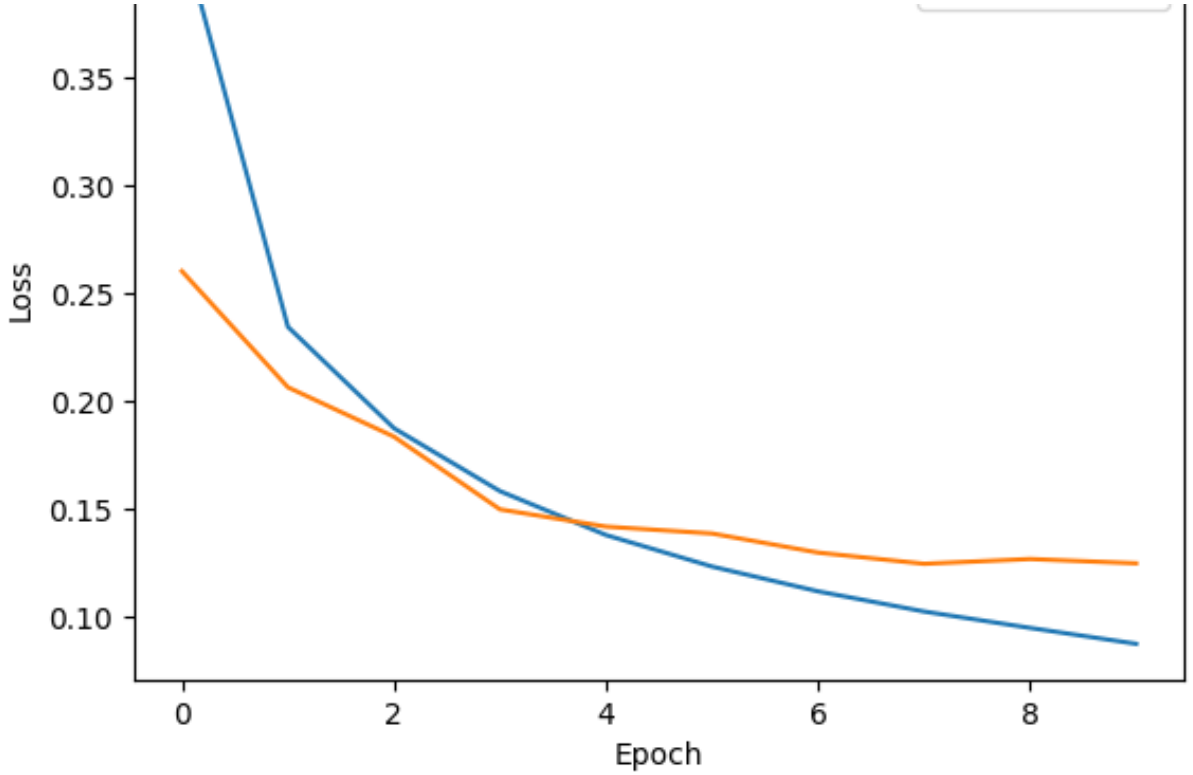
Layer (type)	Output Shape	P
dense_10 (Dense)	(None, 32)	
dense_11 (Dense)	(None, 10)	

Total params: 25,450 (99.41 KB)
Trainable params: 25,450 (99.41 KB)

```
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8055 - loss:
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 7ms/step - accuracy: 0.9300 - loss:
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9443 - loss:
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 6s 6ms/step - accuracy: 0.9545 - loss:
Epoch 5/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 7s 3ms/step - accuracy: 0.9598 - loss:
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9645 - loss:
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9691 - loss:
Epoch 8/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9702 - loss:
Epoch 9/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.9727 - loss:
Epoch 10/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9756 - loss:
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9579 - loss:
Test accuracy: 0.9639000296592712
```







64 Hidden Layers

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images (28x28 → 784)
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# 5. Build model (Added 64 and 32 neuron layers)
```

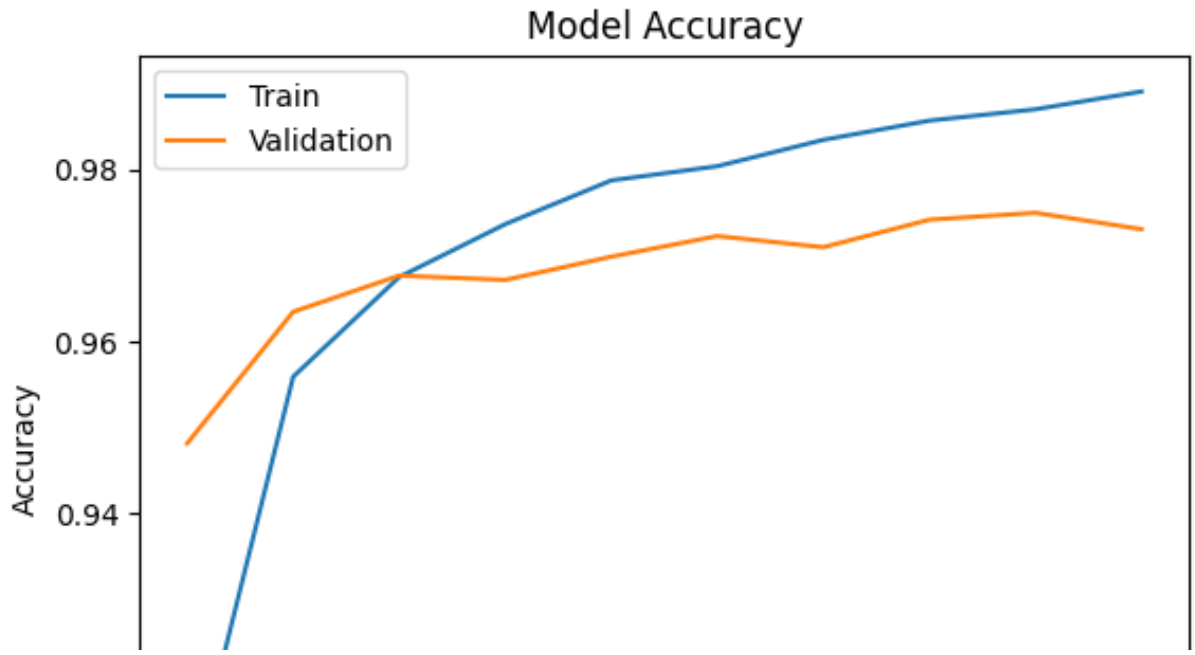
```
model = Sequential([
    Dense(64, activation='relu', input_shape=(784,)),  # First hidd
    Dense(32, activation='relu'),                      # Second hid
    Dense(10, activation='softmax')                    # Output lay
])

# 6. Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 7. Model summary
model.summary()

# 8. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)

# 9. Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# 10. Plot Accuracy Curve
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

# 11. Plot Loss Curve
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```

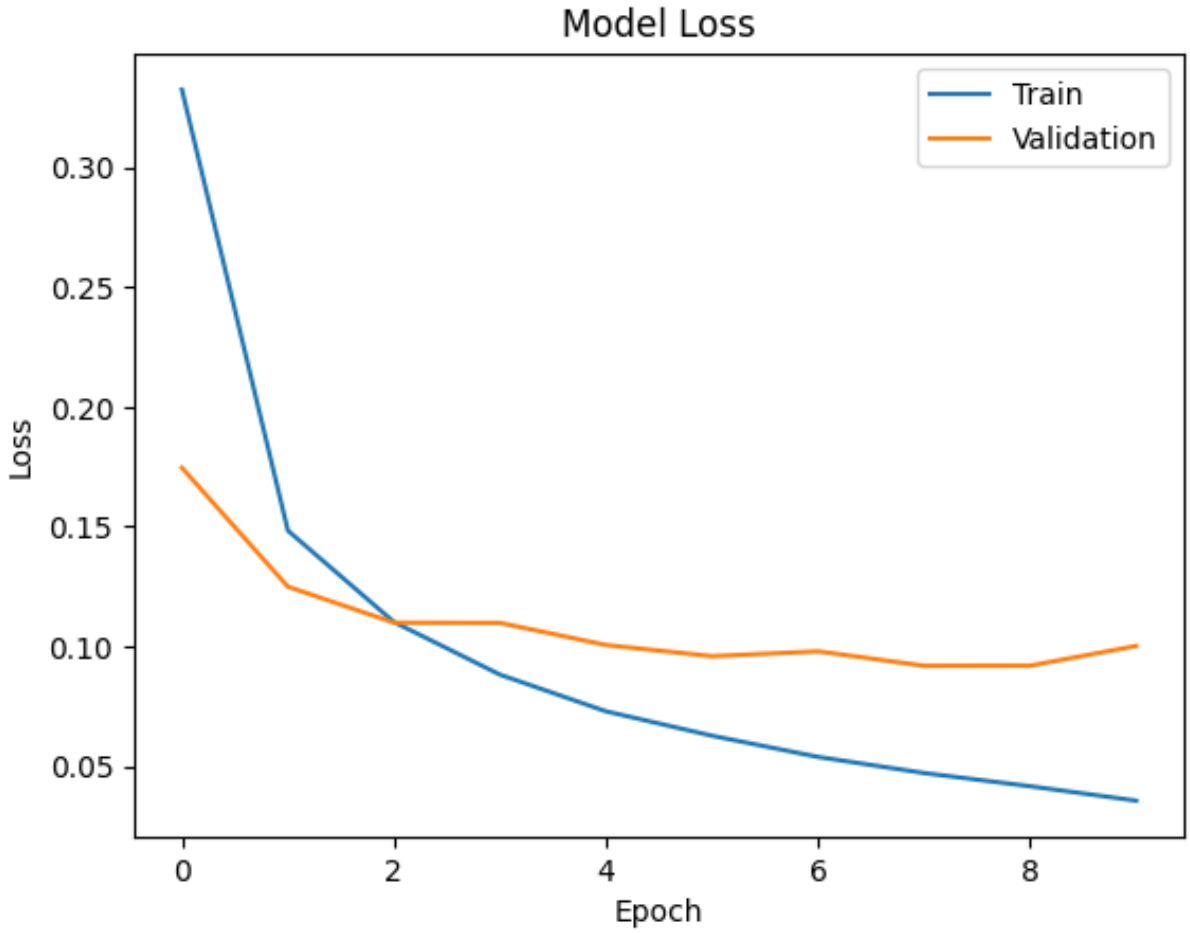
Model: "sequential_11"

Layer (type)	Output Shape	P
dense_12 (Dense)	(None, 64)	
dense_13 (Dense)	(None, 32)	
dense_14 (Dense)	(None, 10)	

Total params: 52,650 (205.66 KB)
Trainable params: 52,650 (205.66 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.8321 - loss:
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9543 - loss:
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9657 - loss:
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9732 - loss:
Epoch 5/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9789 - loss:
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9807 - loss:
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.9845 - loss:
Epoch 8/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9861 - loss:
Epoch 9/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9878 - loss:
Epoch 10/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9899 - loss:
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9695 - loss:
Test accuracy: 0.9729999899864197
```





OCR Detection

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
from google.colab import files

# 1. Load MNIST dataset
```

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images (28x28 → 784)
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# 5. Build neural network
model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(10, activation='softmax')
])

# 6. Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 7. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)

# 8. Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)

# 9. Upload handwritten image
print("Upload your handwritten digit image")
uploaded = files.upload()

# 10. Load image
filename = list(uploaded.keys())[0]
```

```
img = Image.open(filename).convert("L")

# 11. Resize to MNIST format
img = img.resize((28,28))

# 12. Convert to numpy
img_array = np.array(img)

# 13. Invert colors (MNIST uses white digit on black background)
img_array = 255 - img_array

# 14. Normalize
img_array = img_array / 255.0

# 15. Show processed image
plt.imshow(img_array, cmap='gray')
plt.title("Processed Input Image")
plt.axis("off")
plt.show()

# 16. Flatten image
img_array = img_array.reshape(1,784)

# 17. Predict digit
prediction = model.predict(img_array)

digit = np.argmax(prediction)

print("Prediction probabilities:", prediction)
print("Predicted Digit:", digit)
```

```
Epoch 1/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 11ms/step - accuracy: 0.8260 - loss
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9655 - loss:
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9759 - loss:
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9825 - loss:
Epoch 5/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.9847 - loss:
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.9885 - loss:
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9910 - loss:
Epoch 8/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.9919 - loss:
Epoch 9/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9930 - loss:
Epoch 10/10
```

```
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9939 - loss:
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9729 - loss:
Test Accuracy: 0.9753999710083008
Upload your handwritten digit image
```

Choose Files no files selected Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving ChatGPT Image Mar 12, 2026 at 11_11_00 AM.png to ChatGPT Imag
```

Processed Input Image

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 87ms/step
Prediction probabilities: [[1.7705830e-02 3.4021164e-06 1.1809054e-0
  3.7447333e-01 3.7311471e-01 2.7708886e-03 3.7349924e-05 2.3076081e
```

Lab 9

- Shivam Sharma
- 1/23/SET/BCS/424
- 6AIMLB

Handwritten Digit Recognition using CNN

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
from tensorflow.keras.models import Sequential
import tensorflow as tf
```

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0
# reshape for CNN
X_train_cnn = X_train.reshape(-1,28,28,1)
X_test_cnn = X_test.reshape(-1,28,28,1)

model = Sequential([

    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),

    Flatten(),

    Dense(128, activation='relu'),
    Dense(10, activation='softmax')

])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train_cnn, y_train,
          epochs=10,
          batch_size=64,
          validation_split=0.1)

test_loss, test_acc = model.evaluate(X_test_cnn, y_test)

print("Test accuracy:", test_acc)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-k
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolution
  super().__init__(activity_regularizer=activity_regularizer, **kwar
```

```
super().__init__(activity_regularizer=activity_regularizer, kwa
```

Model: "sequential"

Layer (type)	Output Shape	P
conv2d (Conv2D)	(None, 26, 26, 32)	
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	
conv2d_1 (Conv2D)	(None, 11, 11, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	
flatten (Flatten)	(None, 1600)	
dense (Dense)	(None, 128)	2
dense_1 (Dense)	(None, 10)	

```
 Total params: 225,034 (879.04 KB)
 Trainable params: 225,034 (879.04 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 59s 66ms/step - accuracy: 0.9497 - loss
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 75s 58ms/step - accuracy: 0.9839 - loss
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 48s 57ms/step - accuracy: 0.9888 - loss
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 51s 60ms/step - accuracy: 0.9921 - loss
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 49s 58ms/step - accuracy: 0.9943 - loss
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 59ms/step - accuracy: 0.9952 - loss
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 49s 58ms/step - accuracy: 0.9956 - loss
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 59ms/step - accuracy: 0.9972 - loss
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 48s 57ms/step - accuracy: 0.9975 - loss
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 59ms/step - accuracy: 0.9971 - loss
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.9902 - loss:
Test accuracy: 0.9901999831199646
```

ADDED DIFFERENT BATCHES

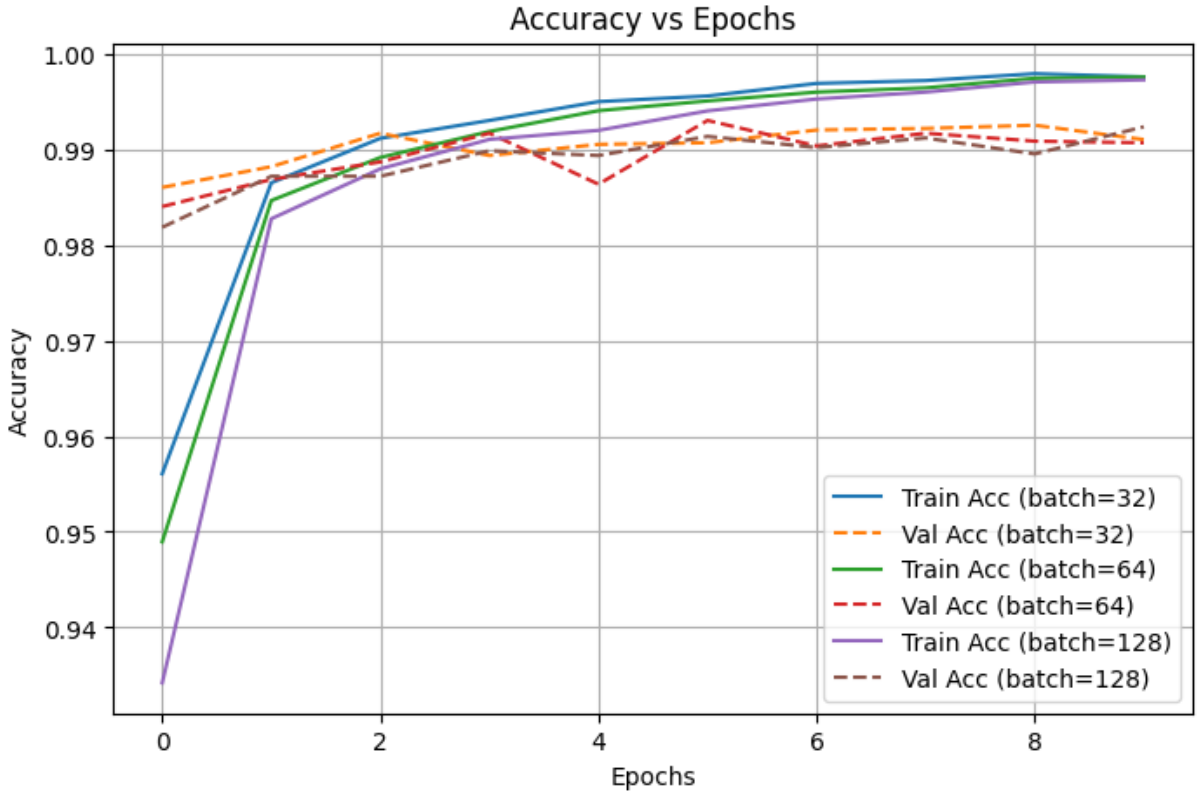
```
import matplotlib.pyplot as plt

batch_sizes = [32, 64, 128]
histories = {}
```

```
for batch in batch_sizes:
    model = Sequential([
        Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1))
        MaxPooling2D((2,2)),

        Conv2D(64, (3,3), activation='relu'),
        MaxPooling2D((2,2)),

        Flatten(),
        Dense(128, activation='relu'),
        Dense(10, activation='softmax')
    ])

    model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

    print(f"\nTraining with batch size: {batch}")
    history = model.fit(X_train_cnn, y_train,
                        epochs=10,
                        batch_size=batch,
                        validation_split=0.1,
                        verbose=1)

    histories[batch] = history

# Plot accuracy
plt.figure(figsize=(8,5))

for batch in batch_sizes:
    plt.plot(histories[batch].history['accuracy'], label=f'Train Ac
    plt.plot(histories[batch].history['val_accuracy'], linestyle='-

plt.title("Accuracy vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()
```

```
Training with batch size: 32
Epoch 1/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 68s 37ms/step - accuracy: 0.9561 - lo
Epoch 2/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 74s 32ms/step - accuracy: 0.9865 - lo
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 54s 32ms/step - accuracy: 0.9911 - lo
```

```
Epoch 4/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 53s 32ms/step - accuracy: 0.9930 - lo
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 53s 31ms/step - accuracy: 0.9950 - lo
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 52s 31ms/step - accuracy: 0.9956 - lo
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 61s 36ms/step - accuracy: 0.9969 - lo
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 73s 31ms/step - accuracy: 0.9972 - lo
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 53s 31ms/step - accuracy: 0.9979 - lo
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 52s 31ms/step - accuracy: 0.9976 - lo

Training with batch size: 64
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 50s 58ms/step - accuracy: 0.9490 - loss
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 47s 55ms/step - accuracy: 0.9846 - loss
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 56ms/step - accuracy: 0.9891 - loss
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 45s 54ms/step - accuracy: 0.9919 - loss
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 55ms/step - accuracy: 0.9940 - loss
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 81s 54ms/step - accuracy: 0.9950 - loss
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 46s 54ms/step - accuracy: 0.9960 - loss
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 50s 59ms/step - accuracy: 0.9964 - loss
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 50s 59ms/step - accuracy: 0.9974 - loss
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 60ms/step - accuracy: 0.9975 - loss

Training with batch size: 128
Epoch 1/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 49s 112ms/step - accuracy: 0.9342 - los
Epoch 2/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 46s 108ms/step - accuracy: 0.9827 - los
Epoch 3/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 45s 106ms/step - accuracy: 0.9879 - los
Epoch 4/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 47s 111ms/step - accuracy: 0.9910 - los
Epoch 5/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 82s 110ms/step - accuracy: 0.9920 - los
Epoch 6/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 80s 106ms/step - accuracy: 0.9940 - los
Epoch 7/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 82s 106ms/step - accuracy: 0.9952 - los
Epoch 8/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 103ms/step - accuracy: 0.9960 - los
```

```
Epoch 9/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 82s 103ms/step - accuracy: 0.9970 - los
Epoch 10/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 44s 104ms/step - accuracy: 0.9972 - los
```

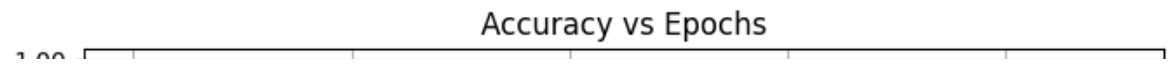


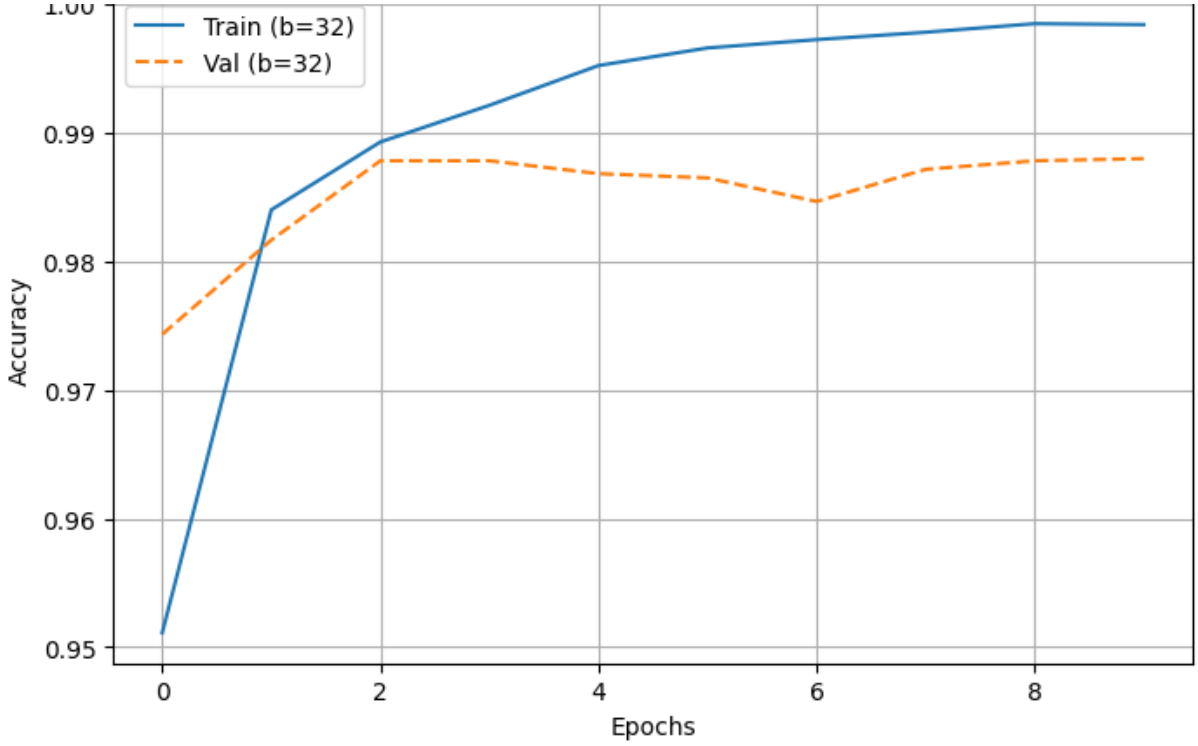
Remove 1 convolutional layer

```
import matplotlib.pyplot as plt

batch_sizes = [32]
histories = {}

for batch in batch_sizes:
    model = Sequential([
        Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1))
        MaxPooling2D((2,2)),

        Flatten(),
        Dense(128, activation='relu'),
        Dense(10, activation='softmax')
    ])
```

```
    model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

    print(f"\nTraining with batch size: {batch}")
    history = model.fit(X_train_cnn, y_train,
                        epochs=10,
                        batch_size=batch,
                        validation_split=0.1,
                        verbose=1)

    histories[batch] = history

# Plot accuracy
plt.figure(figsize=(8,5))

for batch in batch_sizes:
    plt.plot(histories[batch].history['accuracy'], label=f'Train (b
    plt.plot(histories[batch].history['val_accuracy'], linestyle='-

plt.title("Accuracy vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()
```

```
Training with batch size: 32
Epoch 1/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 54s 30ms/step - accuracy: 0.9511 - lo
Epoch 2/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 71s 23ms/step - accuracy: 0.9840 - lo
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 36s 22ms/step - accuracy: 0.9893 - lo
Epoch 4/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 39s 23ms/step - accuracy: 0.9921 - lo
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 39s 23ms/step - accuracy: 0.9952 - lo
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 40s 22ms/step - accuracy: 0.9966 - lo
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 37s 22ms/step - accuracy: 0.9973 - lo
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 42s 23ms/step - accuracy: 0.9978 - lo
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 40s 22ms/step - accuracy: 0.9985 - lo
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 38s 23ms/step - accuracy: 0.9984 - lo
```



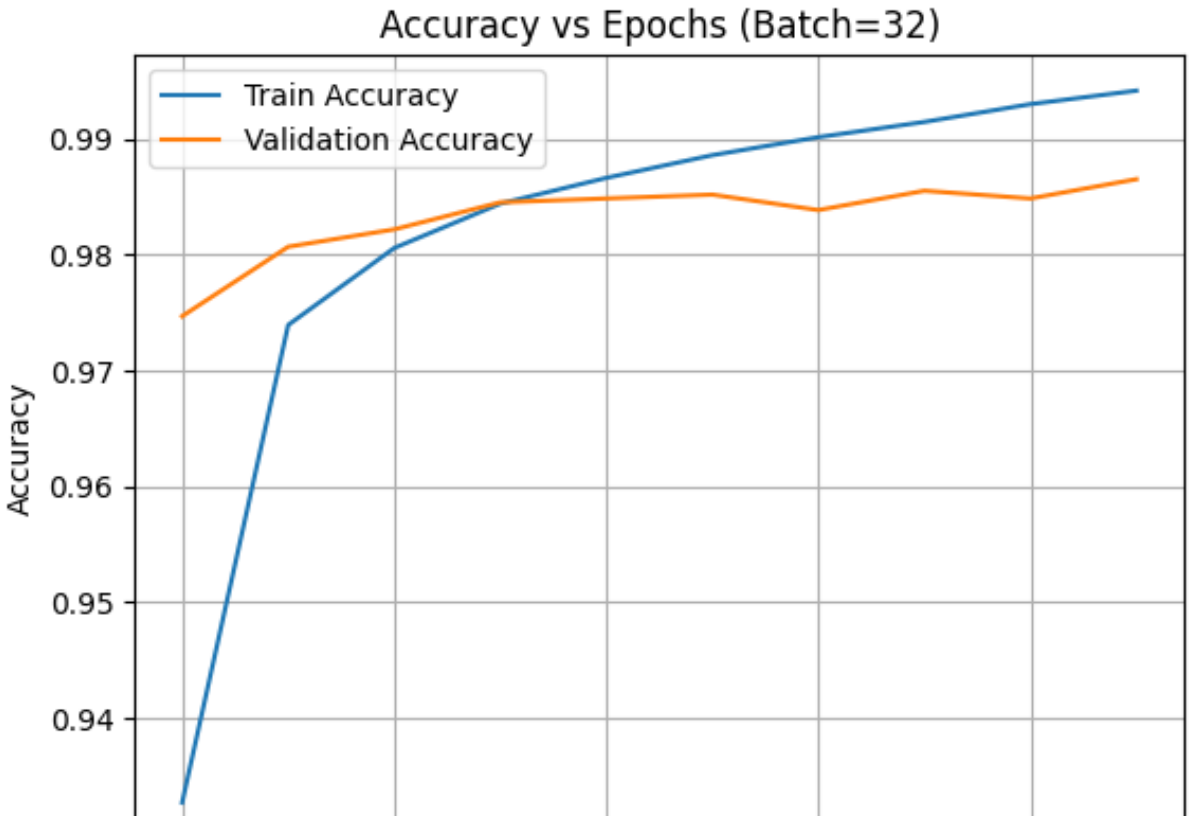


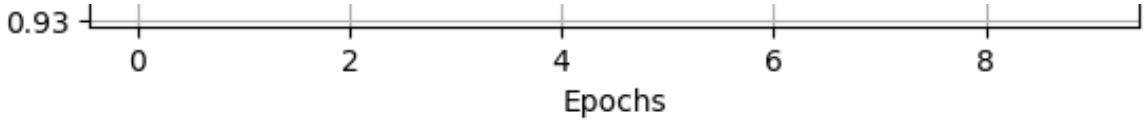
Remove 1 dense layer

```
import matplotlib.pyplot as plt

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Flatten(),
    Dense(10, activation='softmax')   # only one dense layer now
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(X_train_cnn, y_train,
                    epochs=10,
                    batch_size=32,
                    validation_split=0.1)

# Plot accuracy
```

```
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accurac

plt.title("Accuracy vs Epochs (Batch=32)")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()
```

```
Epoch 1/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 30s 17ms/step - accuracy: 0.9327 - lo
Epoch 2/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 28s 17ms/step - accuracy: 0.9739 - lo
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 28s 17ms/step - accuracy: 0.9806 - lo
Epoch 4/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 41s 17ms/step - accuracy: 0.9844 - lo
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 41s 17ms/step - accuracy: 0.9866 - lo
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 28s 17ms/step - accuracy: 0.9886 - lo
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 28s 17ms/step - accuracy: 0.9901 - lo
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 28s 17ms/step - accuracy: 0.9914 - lo
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 30s 18ms/step - accuracy: 0.9930 - lo
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 41s 18ms/step - accuracy: 0.9941 - lo
```





Lab 11

- Shivam Sharma
- 1/23/SET/BCS/424
- 6AIMLB

Recurring Neural Network

Shrinking and Exploding Gradients

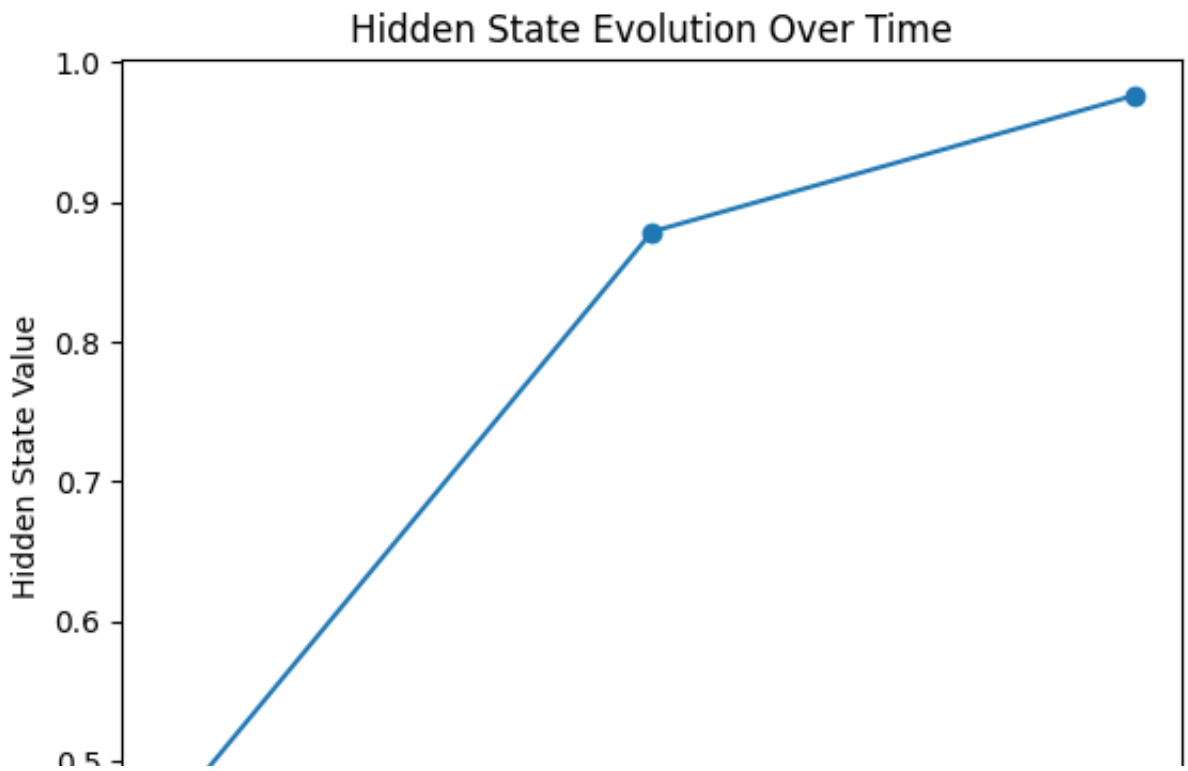
1) Basic RNN

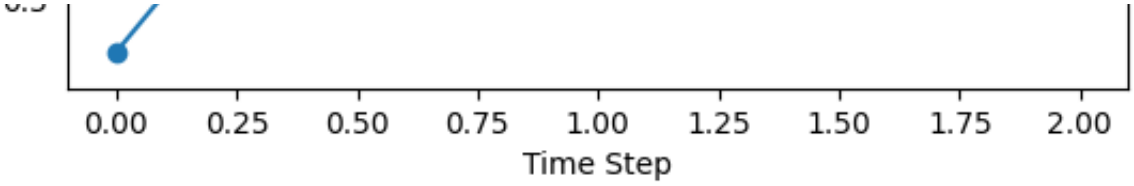
Sample Code

```
import numpy as np
# Sequence: x1, x2, x3
X = [1.0, 2.0, 3.0]
# Weight matrices (scalar case for simplicity)
Wx = 0.5   # input weight
Wh = 0.8   # hidden weight
b = 0.0    # bias

# Initial hidden state
h_prev = 0.0
def tanh(x):
    return np.tanh(x)
hidden_states = []

print("Step-by-step RNN computation:\n")

for t, x_t in enumerate(X):
    h_t = tanh(Wx * x_t + Wh * h_prev + b)

    print(f"Time step {t+1}:")
    print(f"Input x_{t+1} = {x_t}")
```

```
    print(f"h_{t+1} = tanh({Wx}*{x_t} + {Wh}*{h_prev}) = {h_t:.4f}\

    hidden_states.append(h_t)
    h_prev = h_t
    print("Final Hidden States:", hidden_states)

import matplotlib.pyplot as plt

plt.plot(hidden_states, marker='o')
plt.title("Hidden State Evolution Over Time")
plt.xlabel("Time Step")
plt.ylabel("Hidden State Value")
plt.show()
```

```
Step-by-step RNN computation:

Time step 1:
Input x_1 = 1.0
h_1 = tanh(0.5*1.0 + 0.8*0.0) = 0.4621

Final Hidden States: [np.float64(0.46211715726000974)]
Time step 2:
Input x_2 = 2.0
h_2 = tanh(0.5*2.0 + 0.8*0.46211715726000974) = 0.8786

Final Hidden States: [np.float64(0.46211715726000974), np.float64(0.
Time step 3:
Input x_3 = 3.0
h_3 = tanh(0.5*3.0 + 0.8*0.8786223746838898) = 0.9759

Final Hidden States: [np.float64(0.46211715726000974), np.float64(0.
```



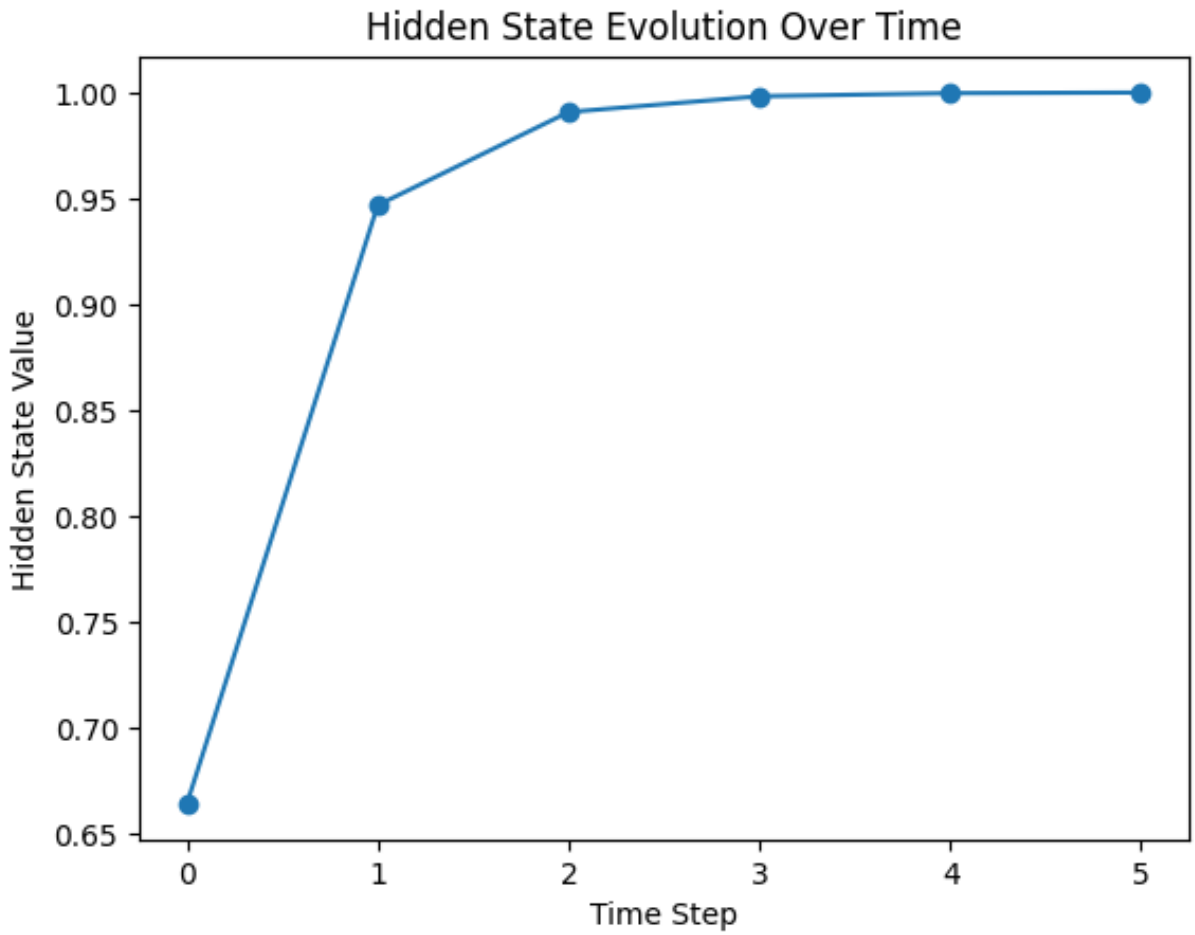


After changing input and weight

```
import numpy as np
# Sequence: x1, x2, x3
X = [1.0, 2.0, 3.0, 4.0, 5.0, 6.0]
# Weight matrices (scalar case for simplicity)
Wx = 0.8   # input weight
Wh = 0.3   # hidden weight
b = 0.0    # bias

# Initial hidden state
h_prev = 0.0
def tanh(x):
    return np.tanh(x)
hidden_states = []

print("Step-by-step RNN computation:\n")

for t, x_t in enumerate(X):
    h_t = tanh(Wx * x_t + Wh * h_prev + b)

    print(f"Time step {t+1}:")
    print(f"Input x_{t+1} = {x_t}")
    print(f"h_{t+1} = tanh({Wx}*{x_t} + {Wh}*{h_prev}) = {h_t:.4f}\

    hidden_states.append(h_t)
    h_prev = h_t
    print("Final Hidden States:", hidden_states)

import matplotlib.pyplot as plt

plt.plot(hidden_states, marker='o')
plt.title("Hidden State Evolution Over Time")
plt.xlabel("Time Step")
plt.ylabel("Hidden State Value")
plt.show()
```

```
Step-by-step RNN computation:

Time step 1:
Input x_1 = 1.0
h_1 = tanh(0.8*1.0 + 0.3*0.0) = 0.6640
```

```
h_1 = tanh(0.8*1.0 + 0.3*0.0) = 0.6640

Final Hidden States: [np.float64(0.664036770267849)]
Time step 2:
Input x_2 = 2.0
h_2 = tanh(0.8*2.0 + 0.3*0.664036770267849) = 0.9467

Final Hidden States: [np.float64(0.664036770267849), np.float64(0.94
Time step 3:
Input x_3 = 3.0
h_3 = tanh(0.8*3.0 + 0.3*0.9467242474459525) = 0.9907

Final Hidden States: [np.float64(0.664036770267849), np.float64(0.94
Time step 4:
Input x_4 = 4.0
h_4 = tanh(0.8*4.0 + 0.3*0.9907167150787952) = 0.9982

Final Hidden States: [np.float64(0.664036770267849), np.float64(0.94
Time step 5:
Input x_5 = 5.0
h_5 = tanh(0.8*5.0 + 0.3*0.9981677295601429) = 0.9996

Final Hidden States: [np.float64(0.664036770267849), np.float64(0.94
Time step 6:
Input x_6 = 6.0
h_6 = tanh(0.8*6.0 + 0.3*0.9996314513187315) = 0.9999

Final Hidden States: [np.float64(0.664036770267849), np.float64(0.94
```



2) BPTT Implementation

```
import numpy as np

# -------------------------
# 1. Input sequence
# -------------------------
X = [1.0, 2.0, 3.0]
target = 0.5   # desired output

# -------------------------
# 2. Initialize parameters
# -------------------------
Wx = 0.5
Wh = 0.8
Wy = 1.0
b = 0.0

# Learning rate
lr = 0.01

# -------------------------
# 3. Activation functions
# -------------------------
def tanh(x):
    return np.tanh(x)

def dtanh(x):
    return 1 - np.tanh(x)**2

# -------------------------
# 4. Forward Pass
# -------------------------
h = [0.0]   # h0 = 0

for t in range(len(X)):
    h_t = tanh(Wx * X[t] + Wh * h[t] + b)
    h.append(h_t)

# Output (many-to-one)
y_pred = Wy * h[-1]

# Loss (Mean Squared Error)
```

```
loss = 0.5 * (y_pred - target)**2

print("Forward Pass:")
print("Hidden states:", h[1:])
print("Prediction:", y_pred)
print("Loss:", loss)

# -------------------------
# 5. Backward Pass (BPTT)
# -------------------------

# Gradients initialization
dWx = 0
dWh = 0
dWy = 0
db = 0

# Output gradient
dy = (y_pred - target)

# Gradient for Wy
dWy = dy * h[-1]

# Backprop through time
dh_next = dy * Wy   # gradient flowing into last hidden state

for t in reversed(range(len(X))):

    # derivative of tanh
    dt = dtanh(Wx * X[t] + Wh * h[t] + b)

    # current gradient
    dh = dh_next * dt
    print(f"\nTime step {t}")
    print("dh:", dh)
    # accumulate gradients
    dWx += dh * X[t]
    dWh += dh * h[t]
    db  += dh

    # pass gradient backward in time
    dh_next = dh * Wh
      # PRINT AFTER UPDATING dh_next
    print("dh_next:", dh_next)
print("\nGradients:")
print("dWx:", dWx)
print("dWh:", dWh)
print("dWy:", dWy)
```

```
print("db:", db)

# -------------------------
# 6. Update weights
# -------------------------
Wx -= lr * dWx
Wh -= lr * dWh
Wy -= lr * dWy
b  -= lr * db

print("\nUpdated Parameters:")
print("Wx:", Wx)
print("Wh:", Wh)
print("Wy:", Wy)
print("b:", b)
```

```
Forward Pass:
Hidden states: [np.float64(0.46211715726000974), np.float64(0.878622
Prediction: 0.9758816208890569
Loss: 0.11323165854999806

Time step 2
dh: 0.02267816815333795
dh_next: 0.01814253452267036

Time step 1
dh: 0.0041369101186258935
dh_next: 0.003309528094900715

Time step 0
dh: 0.0026027708674217116
dh_next: 0.0020822166939373694

Gradients:
dWx: 0.07891109556468734
dWh: 0.02183728310022592
dWy: 0.4644041275445246
db: 0.029417849139385555

Updated Parameters:
Wx: 0.4992108890443531
Wh: 0.7997816271689978
Wy: 0.9953559587245547
b: -0.00029417849139385554
```

3) Application of RNN using IMDB

```
from tensorflow.keras.datasets import imdb
```

```
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense

vocab_size = 3000        # reduced
max_len = 100           # reduced
embed_size = 16         # reduced
hidden_size = 32        # reduced

(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=voc

X_train = pad_sequences(X_train, maxlen=max_len)
X_test = pad_sequences(X_test, maxlen=max_len)

model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=embed_size, input_le
    SimpleRNN(hidden_size),
    Dense(1, activation='sigmoid')
])

model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

model.fit(
    X_train, y_train,
    epochs=20,
    batch_size=32,
    verbose=1
)

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Accuracy: {accuracy*100:.2f}%")
```

```
Epoch 1/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 23ms/step - accuracy: 0.6584 - loss
Epoch 2/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 17s 21ms/step - accuracy: 0.8128 - loss
Epoch 3/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 21ms/step - accuracy: 0.7777 - loss
Epoch 4/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 20ms/step - accuracy: 0.8570 - loss
Epoch 5/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 20ms/step - accuracy: 0.8900 - loss
Epoch 6/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 20ms/step - accuracy: 0.9097 - loss
Epoch 7/20
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 20ms/step - accuracy: 0.9317 - loss
Epoch 8/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 20ms/step - accuracy: 0.9472 - loss
Epoch 9/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 22s 22ms/step - accuracy: 0.9598 - loss
Epoch 10/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 20ms/step - accuracy: 0.9741 - loss
Epoch 11/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 20ms/step - accuracy: 0.9752 - loss
Epoch 12/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 21ms/step - accuracy: 0.9781 - loss
Epoch 13/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 20ms/step - accuracy: 0.9789 - loss
Epoch 14/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 20ms/step - accuracy: 0.9768 - loss
Epoch 15/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 20ms/step - accuracy: 0.9832 - loss
Epoch 16/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 17s 22ms/step - accuracy: 0.9823 - loss
Epoch 17/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 20ms/step - accuracy: 0.9892 - loss
Epoch 18/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 20ms/step - accuracy: 0.9940 - loss
Epoch 19/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 20ms/step - accuracy: 0.9719 - loss
Epoch 20/20
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 21ms/step - accuracy: 0.9941 - loss
Test Accuracy: 75.01%
```